Name: Manahil Habib

Reg no: 2023-BSE-042

Section: 5B

Lab 12

Task 0 Lab Setup (Codespace & GH CLI)

All actions below should be executed inside the Codespace shell.

Create Codespace & connect:

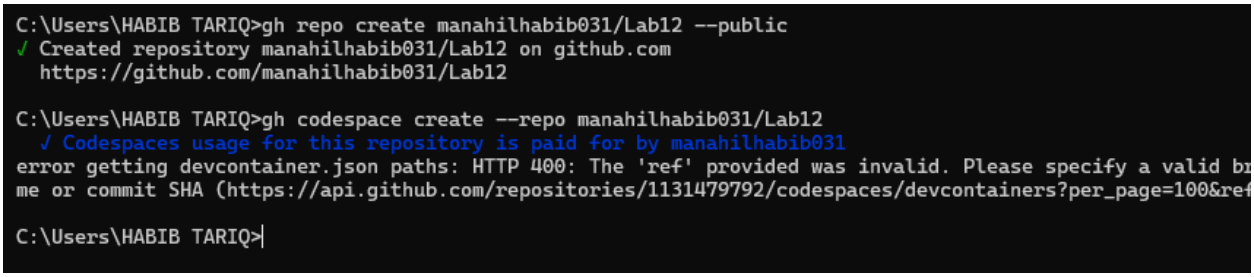
create or open codespace via GH CLI (example)

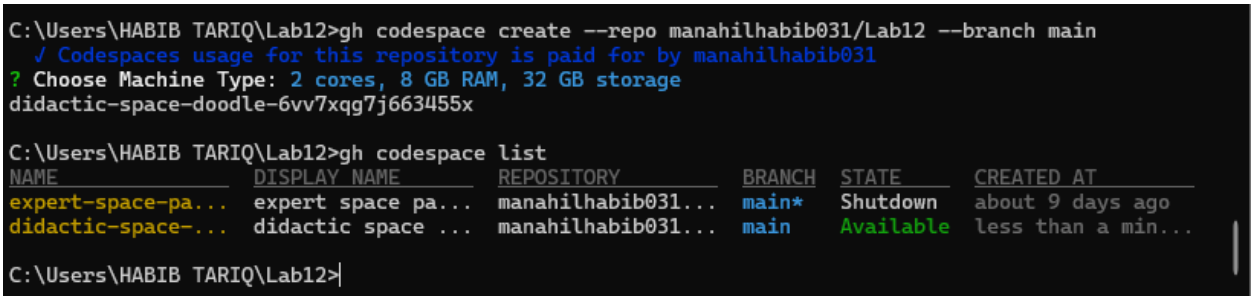
gh repo create CC_<YourName>_<YourRollNumber>/Lab12 --public

gh codespace create --repo <user_name>/Lab12

gh codespace list

gh codespace ssh -c <your_codespace_name>





```
C:\Users\HABIB TARIQ\Lab12>gh codespace ssh -c didactic-space-doodle-6vv7xqg7j663455x
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Task 1 — Organize Terraform code into separate files

In this task, you will split a monolithic Terraform configuration into separate, well-organized files following best practices.

Create the initial project structure:

mkdir -p ~/Lab12

cd ~/Lab12

Create all required files:

touch main.tf variables.tf outputs. tf locals.tf terraform.tfvars entry-script.sh

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ mkdir -p ~/Lab12
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ cd ~/Lab12
@manahilhabib031 ➜ ~/Lab12 $ touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars ent
ry-script.sh
@manahilhabib031 ➜ ~/Lab12 $ ls -la
total 12
drwxrwxr-x 2 codespace codespace 4096 Jan 10 06:39 .
drwxr-x--- 1 codespace codespace 4096 Jan 10 06:38 ..
-rw-rw-r-- 1 codespace codespace    0 Jan 10 06:39 entry-script.sh
-rw-rw-r-- 1 codespace codespace    0 Jan 10 06:39 locals.tf
-rw-rw-r-- 1 codespace codespace    0 Jan 10 06:39 main.tf
-rw-rw-r-- 1 codespace codespace    0 Jan 10 06:39 outputs.tf
-rw-rw-r-- 1 codespace codespace    0 Jan 10 06:39 terraform.tfvars
-rw-rw-r-- 1 codespace codespace    0 Jan 10 06:39 variables.tf
@manahilhabib031 ➜ ~/Lab12 $
```

Create variables.tf with the following content:

variable "vpc_cidr_block" {}

variable "subnet_cidr_block" {}

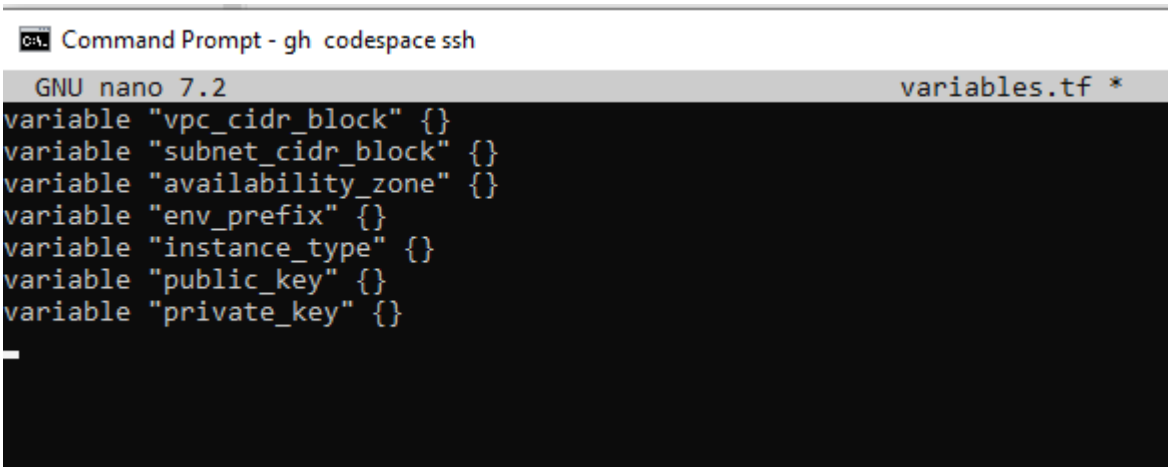
variable "availability_zone" {}

variable "env_prefix" {}

variable "instance_type" {}

variable "public_key" {}

variable "private_key" {}

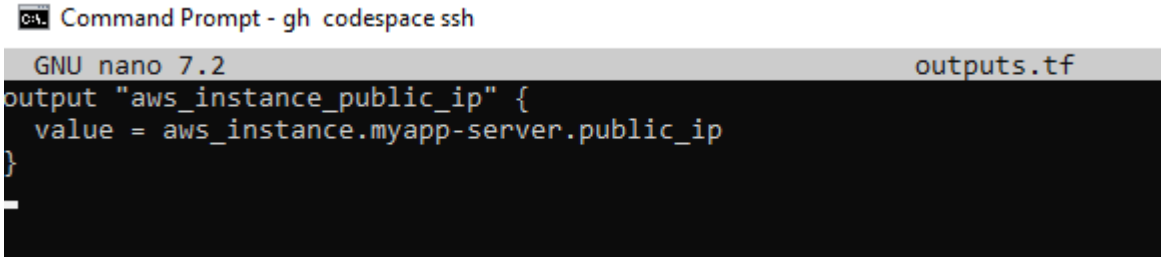


Create outputs.tf with the following content:

output "aws_instance_public_ip" {

  value = aws_instance.myapp-server.public_ip

}

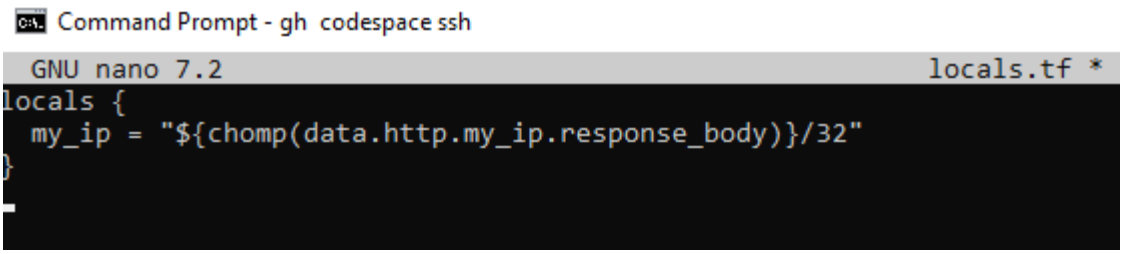


Create locals.tf with the following content:

locals {

  my_ip = "${chomp(data.http.my_ip.response_body)}/32"

}



Create terraform.tfvars with the following content:

vpc_cidr_block = "10.0.0.0/16"

subnet_cidr_block = "10.0.10.0/24"

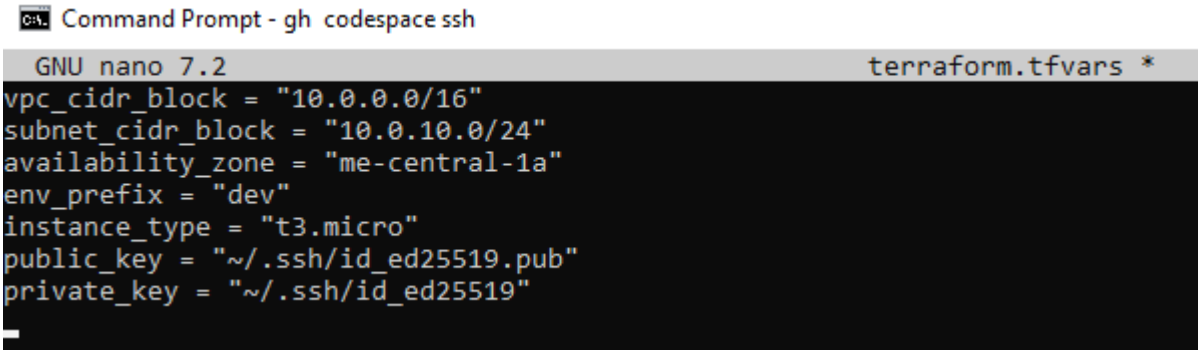
availability_zone = "me-central-1a"

env_prefix = "dev"

instance_type = "t3.micro"

public_key = "~/.ssh/id_ed25519.pub"

private_key = "~/.ssh/id_ed25519"



Create main.tf with the following content:

```
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  tags = {
    Name = "${var.env_prefix}-vpc"
```

```
  }
}
resource "aws_subnet" "myapp_subnet_1" {
  vpc_id     = aws_vpc.myapp_vpc.id
  cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}
resource "aws_default_route_table" "main_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }
    tags = {
     Name = "${var.env_prefix}-rt"
  }
}
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
    tags = {
     Name = "${var.env_prefix}-igw"
  }
}
resource "aws_default_security_group" "default_sg" {
  vpc_id      = aws_vpc.myapp_vpc.id
```

```
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [local.my_ip]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    prefix_list_ids = []
}
  tags = {
     Name = "${var.env_prefix}-default-sg"
  }
}
resource "aws_key_pair" "ssh-key" {
  key_name = "serverkey"
  public_key = file(var.public_key)
}
resource "aws_instance" "myapp-server" {
```

```
  ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023 Kernel 6.1 AMI
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg. id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key. key_name
  user_data = file("./entry-script.sh")
  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
data "http" "my_ip" {
  url = "https://icanhazip.com"
}
```

Command Prompt - gh codespace ssh

```
  GNU nano 7.2                                                  main.tf *
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  tags = {
     Name = "${var.env_prefix}-vpc"
  }
}

resource "aws_subnet" "myapp_subnet_1" {
  vpc_id     = aws_vpc.myapp_vpc.id
  cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  tags = {
     Name = "${var.env_prefix}-subnet-1"
  }
}

resource "aws_default_route_table" "main_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Loca
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go
```

Create entry-script.sh with the following content:

#!/bin/bash

set -e

yum update -y

yum install -y nginx

systemctl start nginx

systemctl enable nginx

Command Prompt - gh codespace ssh

```
  GNU nano 7.2                                          entry-script.sh *
#!/bin/bash
set -e
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx
```

Generate SSH key pair if not already exists:

ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""

```
@manahilhabib031 ➜ ~/Lab12 $ ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /home/codespace/.ssh/id_ed25519
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:N8ve1kjLud3f8e3hZ6UMVOZaxCvC99iwWjwW4TRbjk8 codespace@codespaces-2a2d77
The key's randomart image is:
+--[ED25519 256]--+
|             .   |
|            + *  |
|         . o % . |
|          o X E  |
|        S o= &   |
|         o o@ + .|
|          o* O +.|
|         ...*.=.O|
|          ..o..+X|
+----[SHA256]-----+
@manahilhabib031 ➜ ~/Lab12 $
```

Initialize Terraform:

terraform init

```
@manahilhabib031 ➜ ~/Lab12 $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Finding latest version of hashicorp/http...
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 ➜ ~/Lab12 $
```

Apply the configuration:

terraform apply -auto-approve

```
@manahilhabib031 →~/Lab12 $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_default_route_table.main_rt will be created
  + resource "aws_default_route_table" "main_rt" {
      + arn                    = (known after apply)
      + default_route_table_id = (known after apply)
      + id                     = (known after apply)
      + owner_id               = (known after apply)
      + region                 = "me-central-1"
      + route                  = [
          + {
              + cidr_block                = "0.0.0.0/0"
              + gateway_id                = (known after apply)
                # (10 unchanged attributes hidden)
            },
        ]
      + tags                   = {
          + "Name" = "dev-rt"
        }
      + tags_all               = {
          + "Name" = "dev-rt"
        }
      + vpc_id                 = (known after apply)
    }

  # aws_default_security_group.default_sg will be created
  + resource "aws_default_security_group" "default_sg" {
      + arn                    = (known after apply)
      + description            = (known after apply)
      + egress                 = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + from_port        = 0
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "-1"
              + security_groups  = []
```

```
          - delete_on_termination = true -> null
          - device_name           = "/dev/xvda" -> null
          - encrypted             = false -> null
          - iops                  = 3000 -> null
          - tags                  = {} -> null
          - tags_all              = {} -> null
          - throughput            = 125 -> null
          - volume_id             = "vol-08040f0d5adf96412" -> null
          - volume_size           = 8 -> null
          - volume_type           = "gp3" -> null
            # (1 unchanged attribute hidden)
        }
    }

Plan: 1 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ aws_instance_public_ip = "158.252.72.174" -> (known after apply)
aws_instance.myapp-server: Destroying... [id=i-007068755f06bb216]
aws_instance.myapp-server: Still destroying... [id=i-007068755f06bb216, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-007068755f06bb216, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-007068755f06bb216, 00m30s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-007068755f06bb216, 00m40s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-007068755f06bb216, 00m50s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-007068755f06bb216, 01m00s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-007068755f06bb216, 01m10s elapsed]
aws_instance.myapp-server: Destruction complete after 1m10s
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-008af673c43f69f2f]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

aws_instance_public_ip = "51.112.228.59"
```

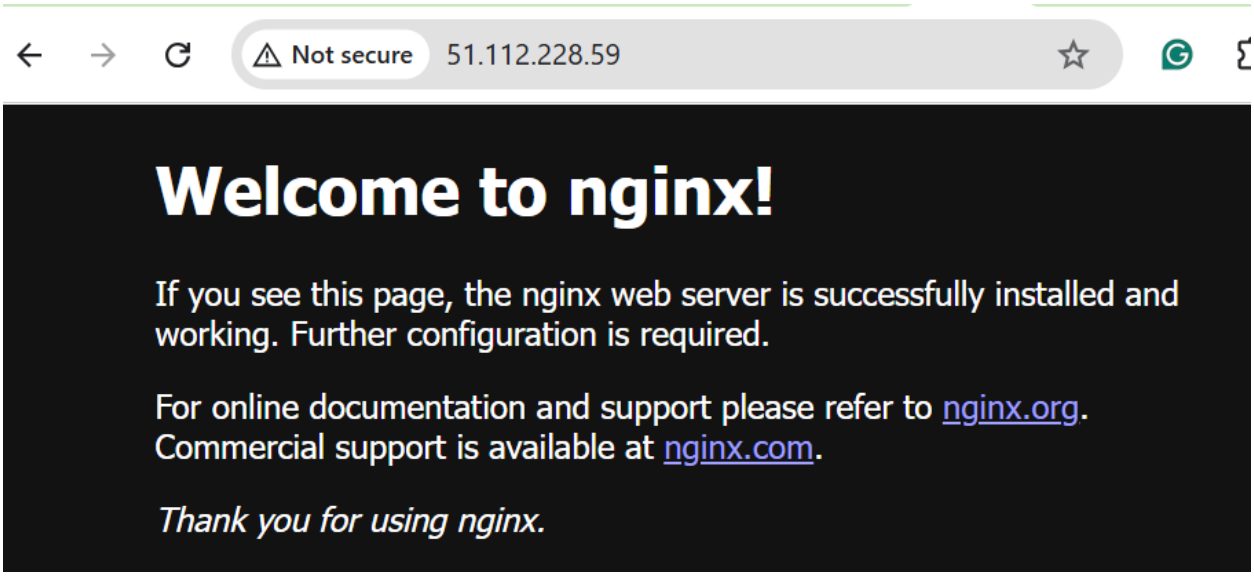
Display the output:

terraform output

```
@manahilhabib031 ➜ ~/Lab12 $ terraform output
aws_instance_public_ip = "51.112.228.59"
@manahilhabib031 ➜ ~/Lab12 $
```

Test nginx in browser:

Open browser and navigate to http://<public-ip>



Destroy resources:

terraform destroy

Type yes when prompted for confirmation.

```
@manahilhabib031 →~/Lab12 $ terraform destroy
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
aws_key_pair.ssh-key: Refreshing state... [id=serverkey]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-09d2a16c5be648f5c]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0e252f3f09b32e79c]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-055c803ccb2f9e0a6]
aws_default_security_group.default_sg: Refreshing state... [id=sg-08b1ed07802e2a341]
aws_default_route_table.main_rt: Refreshing state... [id=rtb-0c23af65427e8110c]
aws_instance.myapp-server: Refreshing state... [id=i-008af673c43f69f2f]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_default_route_table.main_rt will be destroyed
  - resource "aws_default_route_table" "main_rt" {
      - arn                    = "arn:aws:ec2:me-central-1:095032391730:route-table/rtb-0c23af654
27e8110c" -> null
      - default_route_table_id = "rtb-0c23af65427e8110c" -> null
      - id                     = "rtb-0c23af65427e8110c" -> null
      - owner_id               = "095032391730" -> null
      - propagating_vgws       = [] -> null
      - region                 = "me-central-1" -> null
      - route                  = [
          - {
              - cidr_block                = "0.0.0.0/0"
              - gateway_id                = "igw-0e252f3f09b32e79c"
                # (10 unchanged attributes hidden)
            },
        ] -> null
      - tags                   = {
          - "Name" = "dev-rt"
        } -> null
      - tags_all               = {
          - "Name" = "dev-rt"
        } -> null
      - vpc_id                 = "vpc-09d2a16c5be648f5c" -> null
    }

  # aws_default_security_group.default_sg will be destroyed
  - resource "aws_default_security_group" "default_sg" {
      - arn                    = "arn:aws:ec2:me-central-1:095032391730:security-group/sg-08b1ed0
```

```
  - aws_instance_public_ip = "51.112.228.59" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.myapp-server: Destroying... [id=i-008af673c43f69f2f]
aws_default_route_table.main_rt: Destroying... [id=rtb-0c23af65427e8110c]
aws_default_route_table.main_rt: Destruction complete after 0s
aws_internet_gateway.myapp_igw: Destroying... [id=igw-0e252f3f09b32e79c]
aws_instance.myapp-server: Still destroying... [id=i-008af673c43f69f2f, 00m10s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0e252f3f09b32e79c, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-008af673c43f69f2f, 00m20s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0e252f3f09b32e79c, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-008af673c43f69f2f, 00m30s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0e252f3f09b32e79c, 00m30s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-008af673c43f69f2f, 00m40s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0e252f3f09b32e79c, 00m40s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-008af673c43f69f2f, 00m50s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0e252f3f09b32e79c, 00m50s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-008af673c43f69f2f, 01m00s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0e252f3f09b32e79c, 01m00s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-008af673c43f69f2f, 01m10s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-0e252f3f09b32e79c, 01m10s elapsed]
aws_internet_gateway.myapp_igw: Destruction complete after 1m18s
aws_instance.myapp-server: Still destroying... [id=i-008af673c43f69f2f, 01m20s elapsed]
aws_instance.myapp-server: Destruction complete after 1m21s
aws_default_security_group.default_sg: Destroying... [id=sg-08b1ed07802e2a341]
aws_subnet.myapp_subnet_1: Destroying... [id=subnet-055c803ccb2f9e0a6]
aws_key_pair.ssh-key: Destroying... [id=serverkey]
aws_default_security_group.default_sg: Destruction complete after 0s
aws_key_pair.ssh-key: Destruction complete after 0s
aws_subnet.myapp_subnet_1: Destruction complete after 0s
aws_vpc.myapp_vpc: Destroying... [id=vpc-09d2a16c5be648f5c]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
@manahilhabib031 ➜ ~/Lab12 $
```

Task 2 — Use remote-exec provisioner

In this task, you will replace the user_data approach with the remote-exec provisioner to install and configure nginx.

Modify the aws_instance resource in main.tf to use remote-exec provisioner:

Replace the user_data line with the following provisioner block:

resource "aws_instance" "myapp-server" {

ami = "ami-05524d6658fcf35b6"

instance_type = var.instance_type

subnet_id = aws_subnet.myapp_subnet_1.id

security_groups = [aws_default_security_group.default_sg.id]

availability_zone = var.availability_zone

associate_public_ip_address = true

```
key_name = aws_key_pair.ssh-key.key_name

connection {

type = "ssh"

user = "ec2-user"

private_key = file(var.private_key)

host = self.public_ip

}

provisioner "remote-exec" {

inline = [

"sudo yum update -y",

"sudo yum install -y nginx",

"sudo systemctl start nginx",

"sudo systemctl enable nginx"

]

}

tags = {

Name = "${var. env_prefix}-ec2-instance"

}

}
```

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023 Kernel 6.1 AMI
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg. id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key. key_name

  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file(var.private_key)
    host        = self.public_ip
  }

  provisioner "remote-exec" {
    inline = [
      "sudo yum update -y",
      "sudo yum install -y nginx",
      "sudo systemctl start nginx",
      "sudo systemctl enable nginx"
    ]
  }
```

Apply the configuration:

terraform apply -auto-approve \

```
@manahilhabib031 →~/Lab12 $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_default_route_table.main_rt will be created
  + resource "aws_default_route_table" "main_rt" {
      + arn                    = (known after apply)
      + default_route_table_id = (known after apply)
      + id                     = (known after apply)
      + owner_id               = (known after apply)
      + region                 = "me-central-1"
      + route                  = [
          + {
              + cidr_block                = "0.0.0.0/0"
              + gateway_id                = (known after apply)
                # (10 unchanged attributes hidden)
            },
        ]
      + tags                   = {
          + "Name" = "dev-rt"
        }
      + tags_all               = {
          + "Name" = "dev-rt"
        }
      + vpc_id                 = (known after apply)
    }

  # aws_default_security_group.default_sg will be created
  + resource "aws_default_security_group" "default_sg" {
      + arn                    = (known after apply)
      + description            = (known after apply)
      + egress                 = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + from_port        = 0
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "-1"
              + security_groups  = []
              + self             = false
              + to_port          = 0
                # (1 unchanged attribute hidden)
            },
        ]
```

```
aws_instance.myapp-server (remote-exec):   Installing       : generic-logo   6/7
aws_instance.myapp-server (remote-exec):   Installing       : nginx [      ] 7/7
aws_instance.myapp-server (remote-exec):   Installing       : nginx [==    ] 7/7
aws_instance.myapp-server (remote-exec):   Installing       : nginx [===   ] 7/7
aws_instance.myapp-server (remote-exec):   Installing       : nginx [===== ] 7/7
aws_instance.myapp-server (remote-exec):   Installing       : nginx-1:1.28   7/7
aws_instance.myapp-server (remote-exec):   Running scriptlet: nginx-1:1.28   7/7
aws_instance.myapp-server (remote-exec):   Verifying        : generic-logo   1/7
aws_instance.myapp-server (remote-exec):   Verifying        : gperftools-l   2/7
aws_instance.myapp-server (remote-exec):   Verifying        : libunwind-1.   3/7
aws_instance.myapp-server (remote-exec):   Verifying        : nginx-1:1.28   4/7
aws_instance.myapp-server (remote-exec):   Verifying        : nginx-core-1   5/7
aws_instance.myapp-server (remote-exec):   Verifying        : nginx-filesy   6/7
aws_instance.myapp-server (remote-exec):   Verifying        : nginx-mimety   7/7

aws_instance.myapp-server (remote-exec): ====================================
aws_instance.myapp-server (remote-exec): WARNING:
aws_instance.myapp-server (remote-exec):   A newer release of "Amazon Linux" is available.

aws_instance.myapp-server (remote-exec):   Available Versions:

aws_instance.myapp-server (remote-exec):   Version 2023.10.20260105:
aws_instance.myapp-server (remote-exec):     Run the following command to upgrade to 2023.10.2026
0105:

aws_instance.myapp-server (remote-exec):       dnf upgrade --releasever=2023.10.20260105

aws_instance.myapp-server (remote-exec):     Release notes:
aws_instance.myapp-server (remote-exec):      https://docs.aws.amazon.com/linux/al2023/release-no
tes/relnotes-2023.10.20260105.html

aws_instance.myapp-server (remote-exec): ====================================

aws_instance.myapp-server (remote-exec): Installed:
aws_instance.myapp-server (remote-exec):   generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
aws_instance.myapp-server (remote-exec):   gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
aws_instance.myapp-server (remote-exec):   libunwind-1.4.0-5.amzn2023.0.3.x86_64
aws_instance.myapp-server (remote-exec):   nginx-1:1.28.0-1.amzn2023.0.2.x86_64
aws_instance.myapp-server (remote-exec):   nginx-core-1:1.28.0-1.amzn2023.0.2.x86_64
aws_instance.myapp-server (remote-exec):   nginx-filesystem-1:1.28.0-1.amzn2023.0.2.noarch
aws_instance.myapp-server (remote-exec):   nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

aws_instance.myapp-server (remote-exec): Complete!
aws_instance.myapp-server (remote-exec): Created symlink /etc/systemd/system/multi-user.target.wa
nts/nginx.service → /usr/lib/systemd/system/nginx.service.
aws_instance.myapp-server: Creation complete after 1m0s [id=i-024daeeed56a6372a]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "3.28.215.33"
```

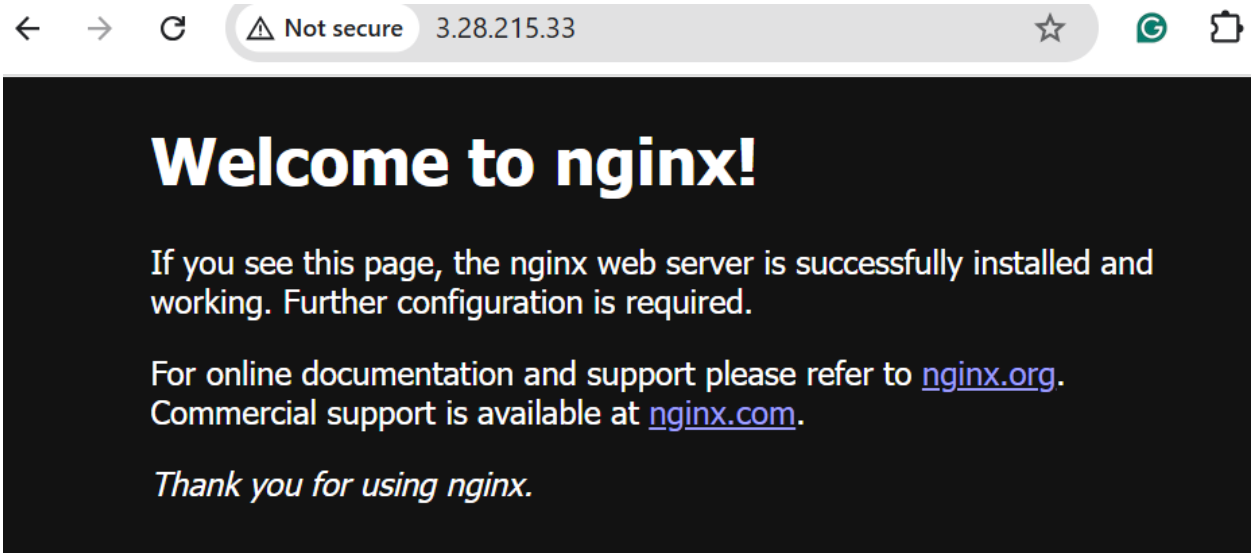
Display the output:

terraform output

```
@manahilhabib031 →~/Lab12 $ terraform output
aws_instance_public_ip = "3.28.215.33"
@manahilhabib031 →~/Lab12 $
```

Test nginx in browser:

Open browser and navigate to http://<public-ip>



Task 3 — Use file and local-exec provisioners

In this task, you will add the file provisioner to upload the script and the local-exec provisioner to log instance information locally.

Modify the aws_instance resource in main.tf to include all three provisioners:

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6"
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp  _subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key.key_name
  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file(var.private_key)
    host        = self.public_ip
  }
```

```
  provisioner "file" {
    source = "./entry-script.sh"
    destination = "/home/ec2-user/entry-script-on-ec2.sh"
  }
  provisioner "remote-exec" {
    inline = [
      "sudo chmod +x /home/ec2-user/entry-script-on-ec2.sh",
      "sudo /home/ec2-user/entry-script-on-ec2.sh"
     ]
  }
  provisioner "local-exec" {
    command = <<-EOF
      echo Instance ${self.id} with public IP ${self.public_ip} has been created
    EOF
  }
  tags = {
     Name = "${var.env_prefix}-ec2-instance"
  }
}
```

Command Prompt - gh codespace ssh

GNU nano 7.2 main.tf *

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6"
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key.key_name

  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file(var.private_key)
    host        = self.public_ip
  }

  provisioner "file" {
    source      = "./entry-script.sh"
    destination = "/home/ec2-user/entry-script-on-ec2.sh"
  }

  provisioner "remote-exec" {
    inline = [
      "sudo chmod +x /home/ec2-user/entry-script-on-ec2.sh",
      "sudo /home/ec2-user/entry-script-on-ec2.sh"
    ]
  }

  provisioner "local-exec" {
    command = <<-EOF
      echo Instance ${self.id} with public IP ${self.public_ip} has been created
    EOF
  }

  tags = {
     Name = "${var.env_prefix}-ec2-instance"
  }
}
```

Apply the configuration:

terraform apply -auto-approve

```
@manahilhabib031 →~/Lab12 $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
aws_key_pair.ssh-key: Refreshing state... [id=serverkey]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-07a9679f95f1a5fd9]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-070048bebe576a1ff]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0fa2b012c059569cd]
aws_default_security_group.default_sg: Refreshing state... [id=sg-0ef94ce653a36f3fc]
aws_default_route_table.main_rt: Refreshing state... [id=rtb-01413079f3c3f3b85]
aws_instance.myapp-server: Refreshing state... [id=i-024daeeed56a6372a]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # aws_instance.myapp-server must be replaced
-/+ resource "aws_instance" "myapp-server" {
      ~ arn                                  = "arn:aws:ec2:me-central-1:095032391730:instance/i-
024daeeed56a6372a" -> (known after apply)
      ~ disable_api_stop                     = false -> (known after apply)
      ~ disable_api_termination              = false -> (known after apply)
      ~ ebs_optimized                        = false -> (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      - hibernation                          = false -> null
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      ~ id                                   = "i-024daeeed56a6372a" -> (known after apply)
      ~ instance_initiated_shutdown_behavior = "stop" -> (known after apply)
      + instance_lifecycle                   = (known after apply)
      ~ instance_state                       = "running" -> (known after apply)
      ~ ipv6_address_count                   = 0 -> (known after apply)
      ~ ipv6_addresses                       = [] -> (known after apply)
      ~ monitoring                           = false -> (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      ~ placement_partition_number           = 0 -> (known after apply)
      ~ primary_network_interface_id         = "eni-0b1bfe7b7bfdce800" -> (known after apply)
      ~ private_dns                          = "ip-10-0-10-13.me-central-1.compute.internal" -> (
known after apply)
      ~ private_ip                           = "10.0.10.13" -> (known after apply)
      + public_dns                           = (known after apply)
```

```
aws_instance.myapp-server (remote-exec):   Verifying        : nginx-core-1   5/7
aws_instance.myapp-server (remote-exec):   Verifying        : nginx-filesy   6/7
aws_instance.myapp-server (remote-exec):   Verifying        : nginx-mimety   7/7
aws_instance.myapp-server (remote-exec): ======================================
aws_instance.myapp-server (remote-exec): WARNING:
aws_instance.myapp-server (remote-exec):   A newer release of "Amazon Linux" is available.

aws_instance.myapp-server (remote-exec):   Available Versions:

aws_instance.myapp-server (remote-exec):   Version 2023.10.20260105:
aws_instance.myapp-server (remote-exec):     Run the following command to upgrade to 2023.10.2026
0105:

aws_instance.myapp-server (remote-exec):       dnf upgrade --releasever=2023.10.20260105

aws_instance.myapp-server (remote-exec):     Release notes:
aws_instance.myapp-server (remote-exec):      https://docs.aws.amazon.com/linux/al2023/release-no
tes/relnotes-2023.10.20260105.html

aws_instance.myapp-server (remote-exec): ======================================

aws_instance.myapp-server (remote-exec): Installed:
aws_instance.myapp-server (remote-exec):   generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
aws_instance.myapp-server (remote-exec):   gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
aws_instance.myapp-server (remote-exec):   libunwind-1.4.0-5.amzn2023.0.3.x86_64
aws_instance.myapp-server (remote-exec):   nginx-1:1.28.0-1.amzn2023.0.2.x86_64
aws_instance.myapp-server (remote-exec):   nginx-core-1:1.28.0-1.amzn2023.0.2.x86_64
aws_instance.myapp-server (remote-exec):   nginx-filesystem-1:1.28.0-1.amzn2023.0.2.noarch
aws_instance.myapp-server (remote-exec):   nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

aws_instance.myapp-server (remote-exec): Complete!
aws_instance.myapp-server (remote-exec): Created symlink /etc/systemd/system/multi-user.target.wa
nts/nginx.service → /usr/lib/systemd/system/nginx.service.
aws_instance.myapp-server: Provisioning with 'local-exec'...
aws_instance.myapp-server (local-exec): Executing: ["/bin/sh" "-c" "echo Instance i-01dd6fe44c895
6a39 with public IP 158.252.72.12 has been created"]
aws_instance.myapp-server (local-exec): Instance i-01dd6fe44c8956a39 with public IP 158.252.72.12
 has been created
aws_instance.myapp-server: Creation complete after 32s [id=i-01dd6fe44c8956a39]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

aws_instance_public_ip = "158.252.72.12"
```

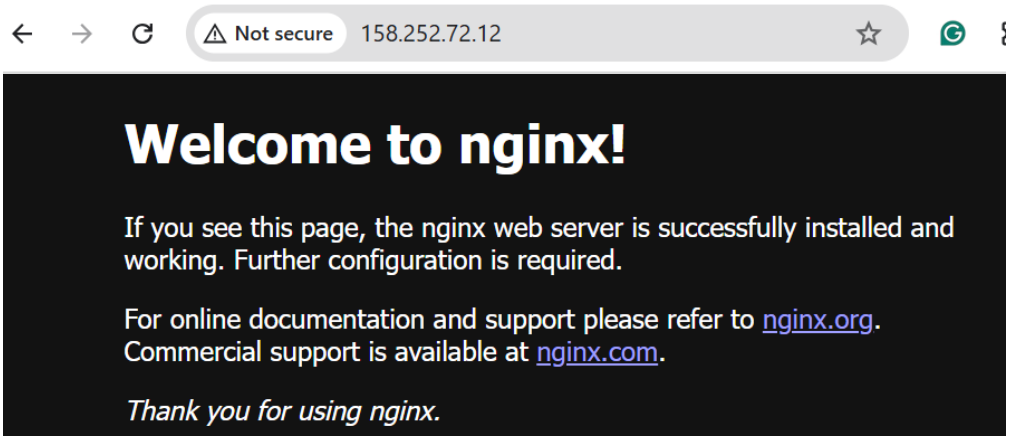
Display the output:

terraform output

```
@manahilhabib031 →~/Lab12 $ terraform output
aws_instance_public_ip = "158.252.72.12"
@manahilhabib031 →~/Lab12 $
```

Test nginx in browser:

Open browser and navigate to http://<public-ip>



Destroy the resources:

terraform destroy

Type yes when prompted.

```
@manahilhabib031 →~/Lab12 $ terraform destroy -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
aws_key_pair.ssh-key: Refreshing state... [id=serverkey]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-07a9679f95f1a5fd9]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-070048bebe576a1ff]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0fa2b012c059569cd]
aws_default_security_group.default_sg: Refreshing state... [id=sg-0ef94ce653a36f3fc]
aws_default_route_table.main_rt: Refreshing state... [id=rtb-01413079f3c3f3b85]
aws_instance.myapp-server: Refreshing state... [id=i-01dd6fe44c8956a39]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_default_route_table.main_rt will be destroyed
  - resource "aws_default_route_table" "main_rt" {
      - arn                    = "arn:aws:ec2:me-central-1:095032391730:route-table/rtb-01413079f
3c3f3b85" -> null
      - default_route_table_id = "rtb-01413079f3c3f3b85" -> null
      - id                     = "rtb-01413079f3c3f3b85" -> null
      - owner_id               = "095032391730" -> null
      - propagating_vgws       = [] -> null
      - region                 = "me-central-1" -> null
      - route                  = [
          - {
              - cidr_block                = "0.0.0.0/0"
              - gateway_id                = "igw-070048bebe576a1ff"
                # (10 unchanged attributes hidden)
            },
        ] -> null
      - tags                   = {
          - "Name" = "dev-rt"
        } -> null
      - tags_all               = {
          - "Name" = "dev-rt"
```

Remove the provisioners and restore user_data:

Replace the connection and provisioner blocks with:

user_data = file("./entry-script.sh")

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6"
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key.key_name

  user_data = file("./entry-script.sh")

  tags = {
     Name = "${var.env_prefix}-ec2-instance"
  }
}


data "http" "my_ip" {
  url = "https://icanhazip.com"
}
```

Task 4 — Create Terraform modules (subnet module)

In this task, you will create a reusable subnet module to organize your infrastructure code better.

Create the module directory structure:

mkdir -p modules/subnet

touch modules/subnet/main.tf

touch modules/subnet/variables.tf

touch modules/subnet/outputs. Tf

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ mkdir -p modules/subnet
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ ouch modules/subnet/main.tf
-bash: ouch: command not found
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ touch modules/subnet/main.tf
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ touch modules/subnet/variables.tf
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ touch modules/subnet/outputs.tf
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ tree modules
modules
└── subnet
    ├── main.tf
    ├── outputs.tf
    └── variables.tf

2 directories, 3 files
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Create modules/subnet/variables.tf:

variable "vpc_id" {}

variable "subnet_cidr_block" {}

variable "availability_zone" {}

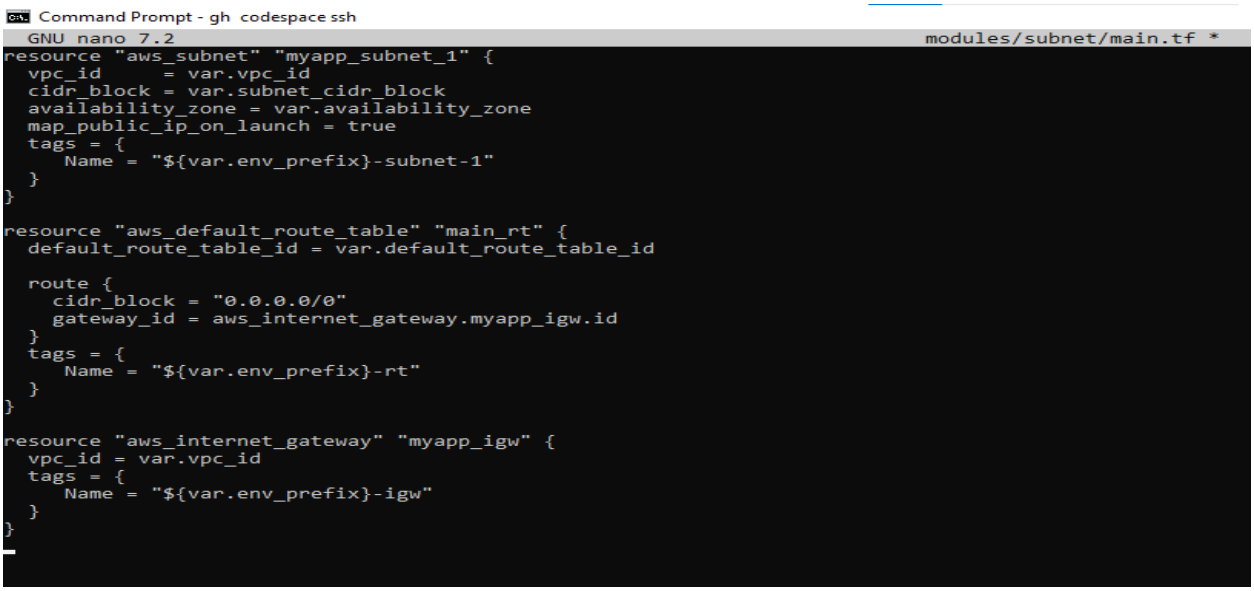
variable "env_prefix" {}

variable "default_route_table_id" {}

Create modules/subnet/main.tf:

```
resource "aws_subnet" "myapp_subnet_1" {
  vpc_id     = var.vpc_id
  cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  map_public_ip_on_launch = true
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}
resource "aws_default_route_table" "main_rt" {
  default_route_table_id = var.default_route_table_id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway. myapp_igw.id
  }
    tags = {
     Name = "${var.env_prefix}-rt"
  }
}
```

```
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = var.vpc_id
    tags = {
     Name = "${var.env_prefix}-igw"
  }
}
```



Create modules/subnet/outputs.tf:

```
output "subnet" {
  value = aws_subnet.myapp_subnet_1
}
```

Modify the root main.tf to use the subnet module:

Remove the subnet, route table, and internet gateway resources and replace them with:

```
module "myapp-subnet" {
  source = "./modules/subnet"
  vpc_id = aws_vpc.myapp_vpc. id
```

subnet_cidr_block = var.subnet_cidr_block

availability_zone = var.availability_zone

env_prefix = var.env_prefix

default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

}

```
}

module "myapp-subnet" {
  source = "./modules/subnet"
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  env_prefix = var.env_prefix
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
}
```

And update the instance resource to reference the module output:

resource "aws_instance" "myapp-server" {

... other settings ...

subnet_id = module.myapp-subnet. subnet.id

... rest of configuration ...

}

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6"
  instance_type = var.instance_type
  subnet_id = module.myapp-subnet.subnet.id
  security_groups = [aws_default_security_group.de
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name = aws_key_pair.ssh-key.key_name

  user_data = file("./entry-script.sh")
```

Initialize Terraform to download the module:

terraform init

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform init
Initializing the backend...
Initializing modules...
- myapp-subnet in modules/subnet
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Finding latest version of hashicorp/http...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Apply the configuration:

terraform apply -auto-approve

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_default_security_group.default_sg will be created
  + resource "aws_default_security_group" "default_sg" {
      + arn                    = (known after apply)
      + description            = (known after apply)
      + egress                 = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + from_port        = 0
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "-1"
              + security_groups  = []
              + self             = false
              + to_port          = 0
                # (1 unchanged attribute hidden)
            },
        ]
      + id                     = (known after apply)
      + ingress                = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + from_port        = 80
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 80
                # (1 unchanged attribute hidden)
            },
          + {
```

```
      + arn                                          = (known after apply)
      + assign_ipv6_address_on_creation              = false
      + availability_zone                            = "me-central-1a"
      + availability_zone_id                         = (known after apply)
      + cidr_block                                   = "10.0.10.0/24"
      + enable_dns64                                 = false
      + enable_resource_name_dns_a_record_on_launch  = false
      + enable_resource_name_dns_aaaa_record_on_launch = false
      + id                                           = (known after apply)
      + ipv6_cidr_block_association_id               = (known after apply)
      + ipv6_native                                  = false
      + map_public_ip_on_launch                      = true
      + owner_id                                     = (known after apply)
      + private_dns_hostname_type_on_launch          = (known after apply)
      + tags                                         = {
          + "Name" = "dev-subnet-1"
        }
      + tags_all                                     = {
          + "Name" = "dev-subnet-1"
        }
      + vpc_id                                       = (known after apply)
    }

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + aws_instance_public_ip = (known after apply)
aws_key_pair.ssh-key: Creating...
aws_vpc.myapp_vpc: Creating...
aws_key_pair.ssh-key: Creation complete after 0s [id=serverkey]
aws_vpc.myapp_vpc: Creation complete after 1s [id=vpc-01bb6ec49c1768434]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Creating...
module.myapp-subnet.aws_internet_gateway.myapp_igw: Creating...
aws_default_security_group.default_sg: Creating...
module.myapp-subnet.aws_internet_gateway.myapp_igw: Creation complete after 1s [id=igw-04f06ba4f9
b2ed04f]
module.myapp-subnet.aws_default_route_table.main_rt: Creating...
module.myapp-subnet.aws_default_route_table.main_rt: Creation complete after 1s [id=rtb-0c6290109
b0917296]
aws_default_security_group.default_sg: Creation complete after 4s [id=sg-09d020ea9dd6d7d5b]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Still creating... [00m10s elapsed]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Creation complete after 12s [id=subnet-061d70d3bd8
36ff8e]
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-0fdbf27d7e522561e]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "40.172.100.36"
```

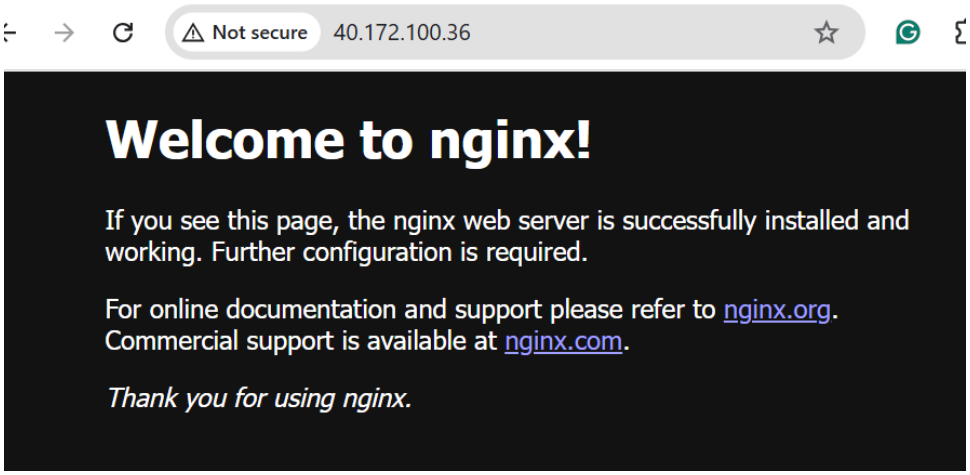
Display the output:

terraform output

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform output
aws_instance_public_ip = "40.172.100.36"
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Test nginx in browser:

Open browser and navigate to http://<public-ip>



Task 5 — Create webserver module

In this task, you will create a reusable webserver module for EC2 instances.

Create the webserver module directory structure:

mkdir -p modules/webserver

touch modules/webserver/main.tf

touch modules/webserver/variables.tf

touch modules/webserver/outputs.tf

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ cd /workspaces/Lab12
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ mkdir -p modules/webserver
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ touch modules/webserver/main.tf modules/webserver/v
ariables.tf modules/webserver/outputs.tf
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ tree module
module  [error opening dir]

0 directories, 0 files
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ tree modules
modules
├── subnet
│   ├── main.tf
│   ├── outputs.tf
│   └── variables.tf
└── webserver
    ├── main.tf
    ├── outputs.tf
    └── variables.tf

3 directories, 6 files
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Create modules/webserver/variables.tf:

variable "env_prefix" {}

variable "instance_type" {}

variable "availability_zone" {}

variable "public_key" {}

variable "my_ip" {}

variable "vpc_id" {}

variable "subnet_id" {}

variable "script_path" {}

variable "instance_suffix" {}

Create modules/webserver/main.tf:

resource "aws_security_group" "web_sg" {

vpc_id = var.vpc_id

name = "${var.env_prefix}-web-sg-${var.instance_suffix}"

description = "Security group for web server allowing HTTP, HTTPS and SSH"

ingress {

from_port = 22

to_port = 22

protocol = "tcp"

cidr_blocks = [var.my_ip]

```
}
ingress {
from_port = 443
to_port = 443
protocol = "tcp"
cidr_blocks = ["0.0.0.0/0"]
}
ingress {
from_port = 80
to_port = 80
protocol = "tcp"
cidr_blocks = ["0.0.0.0/0"]
}
egress {
from_port = 0
to_port = 0
protocol = "-1"
cidr_blocks = ["0.0.0.0/0"]
prefix_list_ids = []
}
tags = {
Name = "${var.env_prefix}-default-sg"
}
```

```
}
resource "aws_key_pair" "ssh-key" {
key_name = "${var.env_prefix}-serverkey-${var.instance_suffix}"
public_key = file(var.public_key)
}
resource "aws_instance" "myapp-server" {
ami = "ami-05524d6658fcf35b6" # Amazon Linux 2023 Kernel 6.1 AMI
instance_type = var. instance_type
subnet_id = var.subnet_id
security_groups = [aws_security_group.web_sg.id]
availability_zone = var. availability_zone
associate_public_ip_address = true
key_name = aws_key_pair.ssh-key.key_name
user_data = file(var.script_path)
tags = {
Name = "${var.env_prefix}-ec2-instance-${var.instance_suffix}"
}
}
```

Command Prompt - gh codespace ssh

```
  GNU nano 7.2                                                  modules/webserver/main.tf *
resource "aws_security_group" "web_sg" {
  vpc_id      = var.vpc_id
  name        = "${var.env_prefix}-web-sg-${var.instance_suffix}"
  description = "Security group for web server allowing HTTP, HTTPS and SSH"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

Create modules/webserver/outputs.tf:

output "aws_instance" {

value = aws_instance.myapp-server

}

Command Prompt - gh codespace ssh

```
  GNU nano 7.2                                                  modules/webserver/outputs.tf *
output "aws_instance" {
  value = aws_instance.myapp-server
}
```

Modify the root main.tf:

Remove the security group, key pair, and instance resources. Replace them with:

module "myapp-webserver" {

source = "./modules/webserver"

env_prefix = var.env_prefix

instance_type = var. instance_type

availability_zone = var.availability_zone

public_key = var.public_key

my_ip = local.my_ip

vpc_id = aws_vpc.myapp_vpc.id

subnet_id = module.myapp-subnet.subnet.id

script_path = "./entry-script.sh"

instance_suffix = "0"

}

Command Prompt - gh codespace ssh

GNU nano 7.2 main.tf *

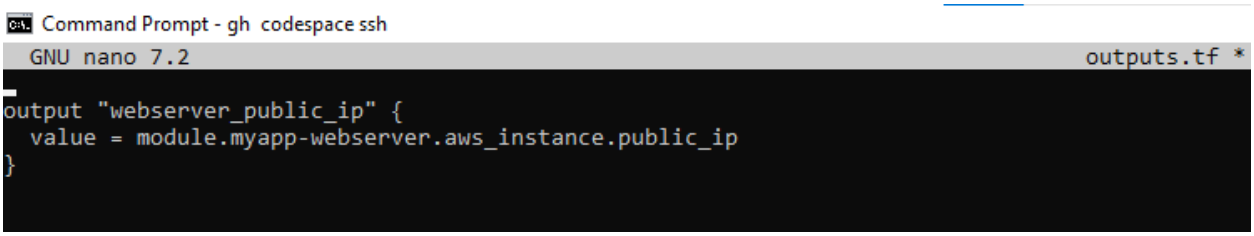
```
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  tags = {
     Name = "${var.env_prefix}-vpc"
  }
}
module "myapp-subnet" {
  source = "./modules/subnet"
  vpc_id = aws_vpc.myapp_vpc. id
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  env_prefix = var.env_prefix
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
}
module "myapp-webserver" {
  source = "./modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_id = module.myapp-subnet.subnet.id
  script_path = "./entry-script.sh"
  instance_suffix = "0"
}

data "http" "my_ip" {
  url = "https://icanhazip.com"
```

Update outputs.tf:

output "webserver_public_ip" {

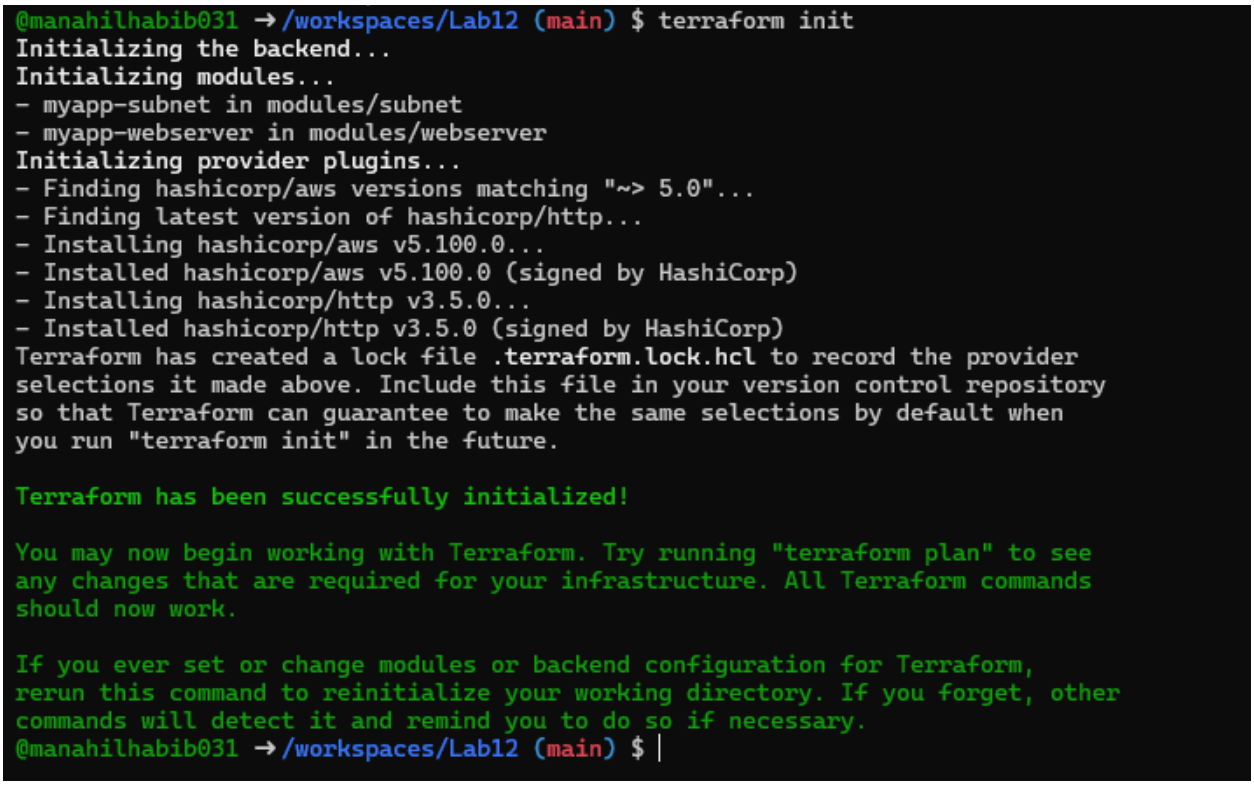
value = module.myapp-webserver.aws_instance.public_ip

}



Initialize Terraform:

terraform init



Apply the configuration:

terraform apply -auto-approve

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
aws_key_pair.ssh-key: Refreshing state... [id=serverkey]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-01bb6ec49c1768434]
aws_instance.myapp-server: Refreshing state... [id=i-0fdbf27d7e522561e]
aws_default_security_group.default_sg: Refreshing state... [id=sg-09d020ea9dd6d7d5b]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-04f06ba4f9b2ed04f
]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-061d70d3bd836ff8e]
module.myapp-subnet.aws_default_route_table.main_rt: Refreshing state... [id=rtb-0c6290109b091729
6]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  + create
  - destroy
```

```
            + from_port        = 22
            + ipv6_cidr_blocks = []
            + prefix_list_ids  = []
            + protocol         = "tcp"
            + security_groups  = []
            + self             = false
            + to_port          = 22
              # (1 unchanged attribute hidden)
          },
      ]
    + name                   = "dev-web-sg-0"
    + name_prefix            = (known after apply)
    + owner_id               = (known after apply)
    + revoke_rules_on_delete = false
    + tags                   = {
        + "Name" = "dev-web-sg-0"
      }
    + tags_all               = {
        + "Name" = "dev-web-sg-0"
      }
    + vpc_id                 = "vpc-01bb6ec49c1768434"
  }

Plan: 3 to add, 0 to change, 3 to destroy.

Changes to Outputs:
  - aws_instance_public_ip = "40.172.100.36" -> null
  + webserver_public_ip    = (known after apply)
aws_instance.myapp-server: Destroying... [id=i-0fdbf27d7e522561e]
module.myapp-webserver.aws_key_pair.ssh_key: Creating...
module.myapp-webserver.aws_security_group.web_sg: Creating...
module.myapp-webserver.aws_key_pair.ssh_key: Creation complete after 0s [id=dev-serverkey-0]
module.myapp-webserver.aws_security_group.web_sg: Creation complete after 2s [id=sg-09d35f44ab31
70f3]
module.myapp-webserver.aws_instance.myapp_server: Creating...
aws_instance.myapp-server: Still destroying... [id=i-0fdbf27d7e522561e, 00m10s elapsed]
module.myapp-webserver.aws_instance.myapp_server: Still creating... [00m10s elapsed]
module.myapp-webserver.aws_instance.myapp_server: Creation complete after 13s [id=i-0371ef3e4977
a437]
aws_instance.myapp-server: Still destroying... [id=i-0fdbf27d7e522561e, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0fdbf27d7e522561e, 00m30s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0fdbf27d7e522561e, 00m40s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0fdbf27d7e522561e, 00m50s elapsed]
aws_instance.myapp-server: Destruction complete after 50s
aws_key_pair.ssh-key: Destroying... [id=serverkey]
aws_default_security_group.default_sg: Destroying... [id=sg-09d020ea9dd6d7d5b]
aws_default_security_group.default_sg: Destruction complete after 0s
aws_key_pair.ssh-key: Destruction complete after 1s

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.

Outputs:

webserver_public_ip = "51.112.178.57"
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Display the output:

terraform output

```
@manahilhabib031 →/workspaces/Lab12 (main) $ terraform output
webserver_public_ip = "51.112.178.57"
@manahilhabib031 →/workspaces/Lab12 (main) $
```

Test nginx in browser:

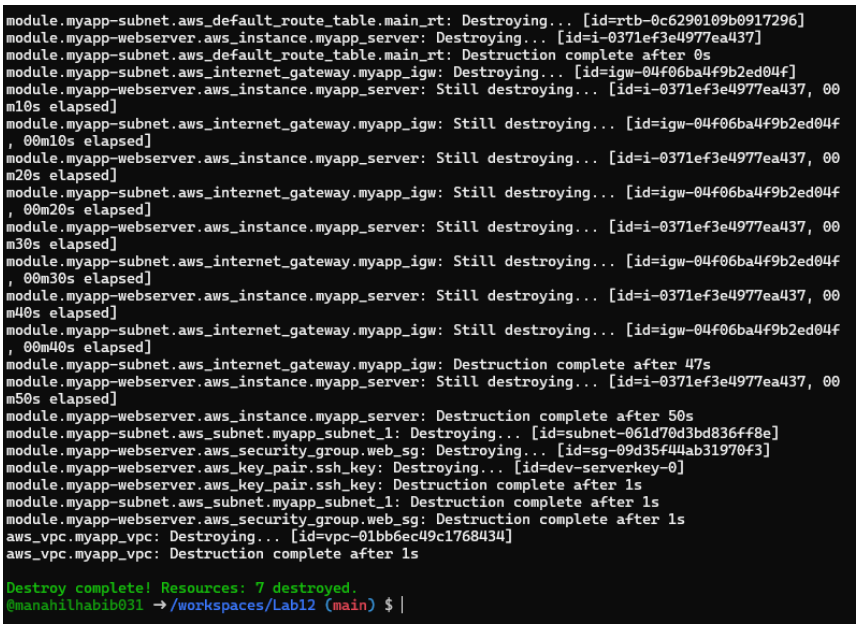
Open browser and navigate to http://<public-ip>

Destroy resources:

terraform destroy

Type yes when prompted.

```
@manahilhabib031 →/workspaces/Lab12 (main) $ terraform destroy
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
module.myapp-webserver.aws_key_pair.ssh_key: Refreshing state... [id=dev-serverkey-0]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-01bb6ec49c1768434]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-04f06ba4f9b2ed04f
]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-061d70d3bd836ff8e]
module.myapp-webserver.aws_security_group.web_sg: Refreshing state... [id=sg-09d35f44ab31970f3]
module.myapp-subnet.aws_default_route_table.main_rt: Refreshing state... [id=rtb-0c6290109b091729
6]
module.myapp-webserver.aws_instance.myapp_server: Refreshing state... [id=i-0371ef3e4977ea437]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # aws_vpc.myapp_vpc will be destroyed
  - resource "aws_vpc" "myapp_vpc" {
      - arn                                  = "arn:aws:ec2:me-central-1:095032391730:vpc/vpc-01b
b6ec49c1768434" -> null
      - assign_generated_ipv6_cidr_block     = false -> null
      - cidr_block                           = "10.0.0.0/16" -> null
      - default_network_acl_id               = "acl-07838153319a3f48b" -> null
      - default_route_table_id               = "rtb-0c6290109b0917296" -> null
      - default_security_group_id            = "sg-09d020ea9dd6d7d5b" -> null
      - dhcp_options_id                      = "dopt-06b319f01537b2a69" -> null
      - enable_dns_hostnames                 = false -> null
      - enable_dns_support                   = true -> null
      - enable_network_address_usage_metrics = false -> null
      - id                                   = "vpc-01bb6ec49c1768434" -> null
      - instance_tenancy                     = "default" -> null
      - ipv6_netmask_length                  = 0 -> null
```



Task 6 — Configure HTTPS with self-signed certificates

In this task, you will configure Nginx to serve traffic over HTTPS using self-signed certificates.

Update entry-script.sh with SSL configuration:

#!/bin/bash

set -e

yum update -y

yum install -y nginx

systemctl start nginx

systemctl enable nginx

Create directories for SSL certificates if they don't exist

mkdir -p /etc/ssl/private

mkdir -p /etc/ssl/certs

```
# Get IMDSv2 token
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
# Get current public IP
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4)
PUBLIC_HOSTNAME=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-hostname)
# Generate self-signed certificate with dynamic IP
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/CN=$PUBLIC_IP" \
-addext "subjectAltName=IP:$PUBLIC_IP" \
-addext "basicConstraints=CA:FALSE" \
-addext "keyUsage=digitalSignature,keyEncipherment" \
-addext "extendedKeyUsage=serverAuth"
echo "Self-signed certificate created for IP: $PUBLIC_IP"
# Backup existing nginx. conf
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
# Overwrite nginx.conf with the desired content
cat <<EOF > /etc/nginx/nginx.conf
user nginx;
```

```
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx. pid;
events {
worker_connections 1024;
}
http {
log_format main '\$remote_addr - \$remote_user [\$time_local] "\$request" '
'\$status \$body_bytes_sent "\$http_referer" '
'"\$http_user_agent" "\$http_x_forwarded_for"';
access_log /var/log/nginx/access.log main;
sendfile on;
tcp_nopush on;
keepalive_timeout 65;
types_hash_max_size 4096;
include /etc/nginx/mime.types;
default_type application/octet-stream;
upstream backend_servers {
server 158.252.94.241:80;
server 158.252.94.242:80 backup;
}
server {
listen 443 ssl;
```

```
server_name $PUBLIC_IP;

ssl_certificate /etc/ssl/certs/selfsigned. crt;

ssl_certificate_key /etc/ssl/private/selfsigned.key;

location / {

root /usr/share/nginx/html;

index index.html;

# proxy_pass http://158.252.94.241:80;

# proxy_pass http://backend_servers;

}

}

server {

listen 80;

server_name _;

return 301 https://\$host\$request_uri;

}

}

EOF

# Test and restart Nginx

systemctl restart nginx
```

Command Prompt - gh codespace ssh

GNU nano 7.2 entry-script.sh *

```
#!/bin/bash
set -e

yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx

# Create directories for SSL certificates
mkdir -p /etc/ssl/private
mkdir -p /etc/ssl/certs

# Get IMDSv2 token
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Get public IP and hostname
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/public-ipv4)

PUBLIC_HOSTNAME=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/public-hostname)

# Generate self-signed certificate
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/selfsigned.key \
  -out /etc/ssl/certs/selfsigned.crt \
  -subj "/CN=$PUBLIC_IP" \
  -addext "subjectAltName=IP:$PUBLIC_IP"

echo "Self-signed certificate created for IP: $PUBLIC_IP"

# Backup nginx.conf
```

Apply the configuration:

terraform apply -auto-approve

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_vpc.myapp_vpc will be created
  + resource "aws_vpc" "myapp_vpc" {
      + arn                                  = (known after apply)
      + cidr_block                           = "10.0.0.0/16"
      + default_network_acl_id               = (known after apply)
      + default_route_table_id               = (known after apply)
      + default_security_group_id            = (known after apply)
      + dhcp_options_id                      = (known after apply)
      + enable_dns_hostnames                 = (known after apply)
      + enable_dns_support                   = true
      + enable_network_address_usage_metrics = (known after apply)
      + id                                   = (known after apply)
      + instance_tenancy                     = "default"
      + ipv6_association_id                  = (known after apply)
      + ipv6_cidr_block                      = (known after apply)
      + ipv6_cidr_block_network_border_group = (known after apply)
      + main_route_table_id                  = (known after apply)
      + owner_id                             = (known after apply)
      + tags                                 = {
          + "Name" = "dev-vpc"
        }
      + tags_all                             = {
          + "Name" = "dev-vpc"
        }
    }

  # module.myapp-subnet.aws_default_route_table.main_rt will be created
  + resource "aws_default_route_table" "main_rt" {
      + arn                    = (known after apply)
      + default_route_table_id = (known after apply)
      + id                     = (known after apply)
```

```
          + "Name" = "dev-web-sg-0"
        }
      + tags_all               = {
          + "Name" = "dev-web-sg-0"
        }
      + vpc_id                 = (known after apply)
    }

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + webserver_public_ip = (known after apply)
module.myapp-webserver.aws_key_pair.ssh_key: Creating...
aws_vpc.myapp_vpc: Creating...
module.myapp-webserver.aws_key_pair.ssh_key: Creation complete after 0s [id=dev-serverkey-0]
aws_vpc.myapp_vpc: Creation complete after 1s [id=vpc-03b2e88256558bdf9]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Creating...
module.myapp-subnet.aws_subnet.myapp_subnet_1: Creating...
module.myapp-webserver.aws_security_group.web_sg: Creating...
module.myapp-subnet.aws_internet_gateway.myapp_igw: Creation complete after 1s [id=igw-017f4f433a
edd27a6]
module.myapp-subnet.aws_default_route_table.main_rt: Creating...
module.myapp-subnet.aws_default_route_table.main_rt: Creation complete after 1s [id=rtb-0d3fd7c18
75b0bd17]
module.myapp-webserver.aws_security_group.web_sg: Creation complete after 3s [id=sg-0a686729a9307
cb13]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Still creating... [00m10s elapsed]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Creation complete after 11s [id=subnet-025e02d89c2
96eaaf]
module.myapp-webserver.aws_instance.myapp_server: Creating...
module.myapp-webserver.aws_instance.myapp_server: Still creating... [00m10s elapsed]
module.myapp-webserver.aws_instance.myapp_server: Creation complete after 13s [id=i-0225475a01a1a
e56f]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

webserver_public_ip = "3.29.21.83"
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Display the output:

terraform output

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform output
webserver_public_ip = "3.29.21.83"
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

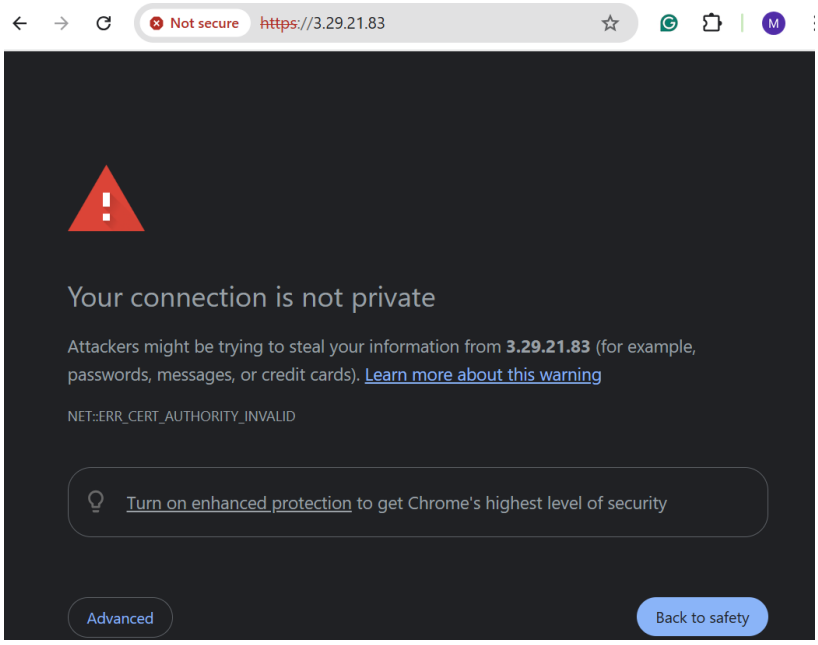
Test HTTPS in browser:

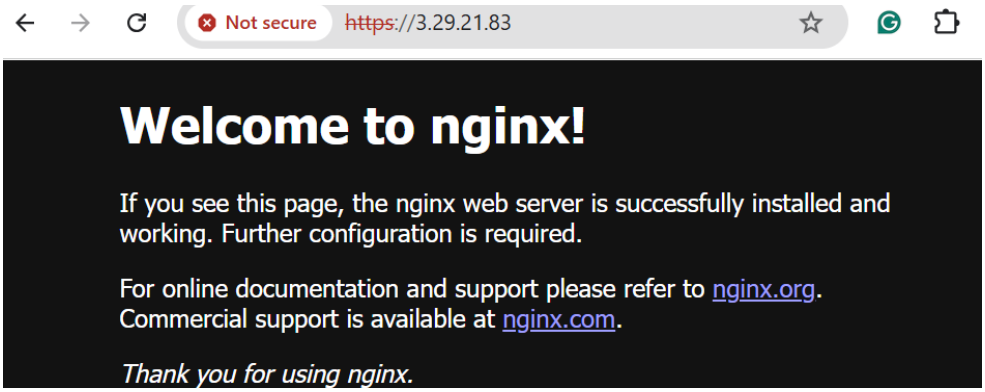
Open browser and navigate to https://<public-ip>

You will see a warning: "Warning: Potential Security Risk Ahead"

Click "Advanced" button

Click "Accept the Risk and Continue"

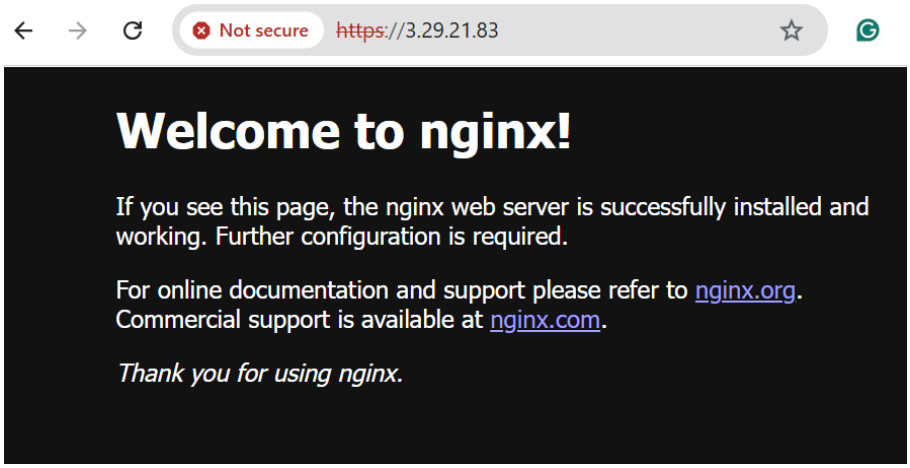




Verify HTTP to HTTPS redirect:

Open browser and navigate to http://<public-ip>

Verify it redirects to https://<public-ip>



Task 7 — Configure Nginx as reverse proxy

In this task, you will create a backend web server and configure Nginx to act as a reverse proxy.

Create apache.sh script for backend web server:

#!/bin/bash

yum update -y

yum install httpd -y

systemctl start httpd

systemctl enable httpd

echo "<h1>Welcome to My Web Server</h1>" > /var/www/html/index.html

hostnamectl set-hostname myapp-webserver

echo "<h2>Hostname: $(hostname)</h2>" >> /var/www/html/index.html

TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \

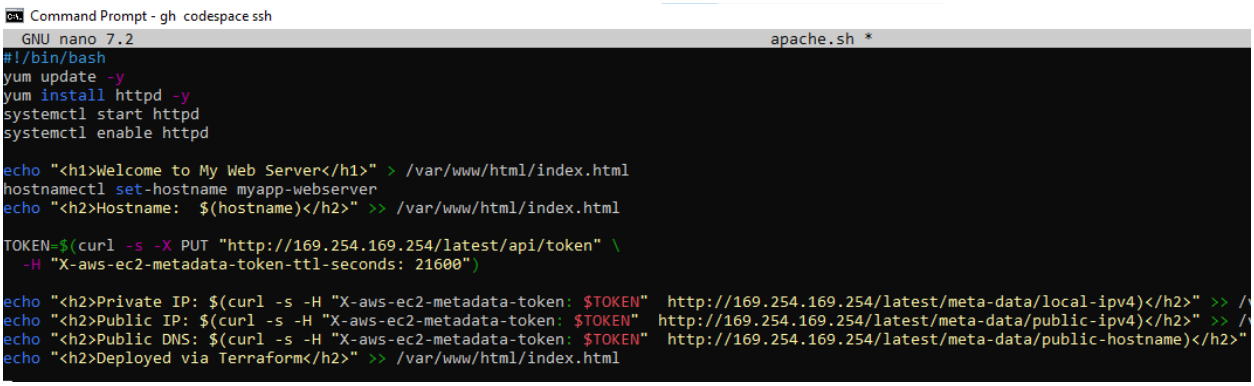
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

echo "<h2>Private IP: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/local-ipv4)</h2>" >> /var/www/html/index.html

echo "<h2>Public IP: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/public-ipv4)</h2>" >> /var/www/html/index.html

echo "<h2>Public DNS: $(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/public-hostname)</h2>" >> /var/www/html/index.html

echo "<h2>Deployed via Terraform</h2>" >> /var/www/html/index. html



Add the backend web server module to main.tf:

module "myapp-web-1" {

source = "./modules/webserver"

env_prefix = var.env_prefix

instance_type = var.instance_type

availability_zone = var.availability_zone

public_key = var. public_key

my_ip = local.my_ip

vpc_id = aws_vpc. myapp_vpc.id

subnet_id = module.myapp-subnet.subnet. id

script_path = "./apache.sh"

instance_suffix = "1"

}

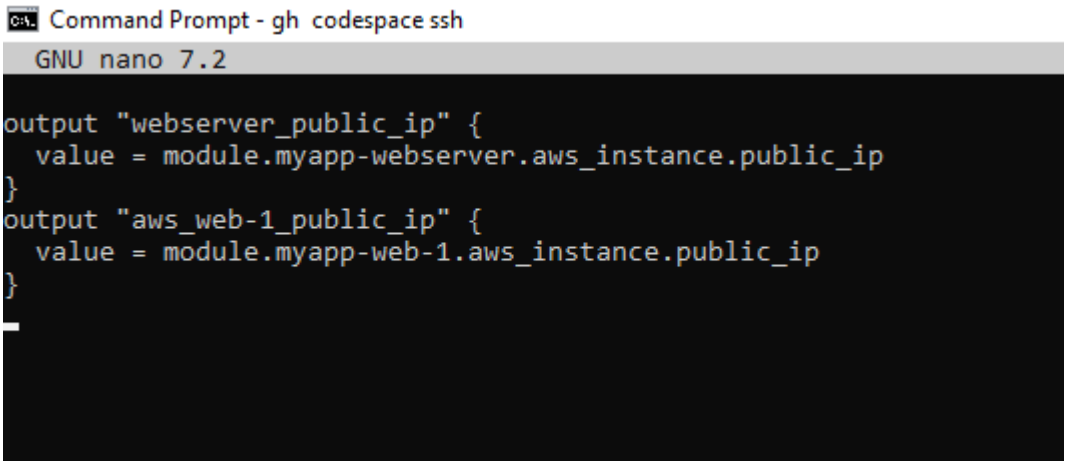
```
}
module "myapp-web-1" {
  source = "./modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_id = module.myapp-subnet.subnet.id
  script_path = "./apache.sh"
  instance_suffix = "1"
}
```

Update outputs.tf:

output "aws_web-1_public_ip" {

value = module.myapp-web-1.aws_instance.public_ip

}



Apply the configuration:

terraform apply -auto-approve

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-03b2e88256558bdf9]
module.myapp-webserver.aws_key_pair.ssh_key: Refreshing state... [id=dev-serverkey-0]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-017f4f433aedd27
]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-025e02d89c296eaa
module.myapp-webserver.aws_security_group.web_sg: Refreshing state... [id=sg-0a686729a9307cb13
module.myapp-subnet.aws_default_route_table.main_rt: Refreshing state... [id=rtb-0d3fd7c1875b0b
7]
module.myapp-webserver.aws_instance.myapp_server: Refreshing state... [id=i-0225475a01a1ae56f]

Terraform used the selected providers to generate the following execution plan. Resource actio
are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # module.myapp-web-1.aws_instance.myapp_server will be created
  + resource "aws_instance" "myapp_server" {
      + ami                                  = "ami-05524d6658fcf35b6"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = true
      + availability_zone                    = "me-central-1a"
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
```

```
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-03b2e88256558bdf9]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Refreshing st
ate... [id=igw-017f4f433aedd27a6]
module.myapp-webserver.aws_security_group.web_sg: Refreshing stat
e... [id=sg-0a686729a9307cb13]
module.myapp-web-1.aws_security_group.web_sg: Refreshing state...
 [id=sg-093bb3320fecccadc]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Refreshing state..
. [id=subnet-025e02d89c296eaaf]
module.myapp-subnet.aws_default_route_table.main_rt: Refreshing s
tate... [id=rtb-0d3fd7c1875b0bd17]
module.myapp-webserver.aws_instance.myapp_server: Refreshing stat
e... [id=i-0225475a01a1ae56f]
module.myapp-web-1.aws_instance.myapp_server: Refreshing state...
 [id=i-0b8cf3c3f1201f519]

Changes to Outputs:
  + webserver_public_ip = "3.29.21.83"

You can apply this plan to save these new output values to the
Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

aws_web_1_public_ip = "51.112.45.114"
webserver_public_ip = "3.29.21.83"
```

Get the outputs:

terraform output

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform output
aws_web_1_public_ip = "51.112.45.114"
webserver_public_ip = "3.29.21.83"
```

SSH into the webserver (Nginx proxy server):

ssh ec2-user@<webserver-public-ip>

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ ssh ec2-user@3.29.21.83
The authenticity of host '3.29.21.83 (3.29.21.83)' can't be established.
ED25519 key fingerprint is SHA256:X33FN6DVAhjEyFHrBlgBln6/tJmOkBybYhEsFMhAu/U.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.29.21.83' (ED25519) to the list of known hosts.

A newer release of "Amazon Linux" is available.
  Version 2023.10.20260105:
Run "/usr/bin/dnf check-release-update" for full release and version update info
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-10-228 ~]$
```

Edit the Nginx configuration:

sudo vim /etc/nginx/nginx.conf

Modify the location block to proxy to web-1:

location / {

root /usr/share/nginx/html;

index index. html;

proxy_pass http://<web-1-public-ip>:80;

proxy_pass http://backend_servers;

}

```
http {
    include       /etc/nginx/mime.types;
    default_type  application/octet-stream;

    sendfile        on;
    keepalive_timeout 65;

    server {
        listen 443 ssl;
        server_name 3.29.21.83;

        ssl_certificate     /etc/ssl/certs/selfsigned.crt;
        ssl_certificate_key /etc/ssl/private/selfsigned.key;

        location / {
            proxy_pass http://51.112.45.114:80
            index index.html;
        }
    }
```

Restart Nginx:

sudo systemctl restart nginx

```
[ec2-user@ip-10-0-10-228 ~]$ sudo journalctl -xeu nginx.service
    Subject: A start job for unit nginx.service has begun executi>
    Defined-By: systemd
    Support: https://lists.freedesktop.org/mailman/listinfo/syste>

    A start job for unit nginx.service has begun execution.

    The job identifier is 20335.
Jan 10 16:01:11 ip-10-0-10-228.me-central-1.compute.internal ngi>
Jan 10 16:01:11 ip-10-0-10-228.me-central-1.compute.internal ngi>
Jan 10 16:01:11 ip-10-0-10-228.me-central-1.compute.internal sys>
    Subject: Unit process exited
    Defined-By: systemd
    Support: https://lists.freedesktop.org/mailman/listinfo/syste>

    An ExecStartPre= process belonging to unit nginx.service has >

    The process' exit code is 'exited' and its exit status is 1.
Jan 10 16:01:11 ip-10-0-10-228.me-central-1.compute.internal sys>
    Subject: Unit failed
    Defined-By: systemd
    Support: https://lists.freedesktop.org/mailman/listinfo/syste>

    The unit nginx.service has entered the 'failed' state with re>
Jan 10 16:01:11 ip-10-0-10-228.me-central-1.compute.internal sys>
```

View Nginx logs and configuration files:

cat /var/log/nginx/error.log

```
[ec2-user@ip-10-0-10-228 ~]$ cat /var/log/nginx/error.log
2026/01/10 08:55:11 [notice] 2929#2929: using the "epoll" event m
ethod
2026/01/10 08:55:11 [notice] 2929#2929: nginx/1.28.0
2026/01/10 08:55:11 [notice] 2929#2929: OS: Linux 6.1.158-180.294
.amzn2023.x86_64
2026/01/10 08:55:11 [notice] 2929#2929: getrlimit(RLIMIT_NOFILE):
 65535:65535
2026/01/10 08:55:11 [notice] 2975#2975: start worker processes
2026/01/10 08:55:11 [notice] 2975#2975: start worker process 2977
2026/01/10 08:55:11 [notice] 2975#2975: start worker process 2978
2026/01/10 08:55:12 [notice] 2975#2975: signal 3 (SIGQUIT) receiv
ed from 1, shutting down
2026/01/10 08:55:12 [notice] 2977#2977: gracefully shutting down
2026/01/10 08:55:12 [notice] 2977#2977: exiting
2026/01/10 08:55:12 [notice] 2977#2977: exit
2026/01/10 08:55:12 [notice] 2978#2978: gracefully shutting down
2026/01/10 08:55:12 [notice] 2978#2978: exiting
2026/01/10 08:55:12 [notice] 2978#2978: exit
2026/01/10 08:55:12 [notice] 2975#2975: signal 17 (SIGCHLD) recei
ved from 2978
2026/01/10 08:55:12 [notice] 2975#2975: worker process 2978 exite
d with code 0
2026/01/10 08:55:12 [notice] 2975#2975: signal 29 (SIGIO) receive
d
2026/01/10 08:55:12 [notice] 2975#2975: signal 17 (SIGCHLD) recei
ved from 2977
2026/01/10 08:55:12 [notice] 2975#2975: worker process 2977 exite
```

cat /var/log/nginx/access.log

```
[ec2-user@ip-10-0-10-228 ~]$ cat /var/log/nginx/access.log
103.147.87.214 - - [10/Jan/2026:08:58:51 +0000] "GET / HTTP/1.1"
200 615 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKi
t/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
103.147.87.214 - - [10/Jan/2026:08:58:51 +0000] "GET /favicon.ico
 HTTP/1.1" 404 555 "https://3.29.21.83/" "Mozilla/5.0 (Windows NT
 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
143.0.0.0 Safari/537.36"
103.147.87.214 - - [10/Jan/2026:08:59:38 +0000] "GET / HTTP/1.1"
301 169 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKi
t/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
103.147.87.214 - - [10/Jan/2026:08:59:39 +0000] "GET / HTTP/1.1"
200 615 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKi
t/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
103.147.87.214 - - [10/Jan/2026:08:59:43 +0000] "GET / HTTP/1.1"
200 615 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKi
t/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
103.147.87.214 - - [10/Jan/2026:08:59:44 +0000] "GET / HTTP/1.1"
200 615 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKi
t/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36"
185.16.39.146 - - [10/Jan/2026:09:02:13 +0000] "GET / HTTP/1.1" 3
01 169 "-" "Wget"
20.65.194.43 - - [10/Jan/2026:09:05:58 +0000] "GET /manager/html
HTTP/1.1" 400 255 "-" "Mozilla/5.0 zgrab/0.x"
185.16.39.146 - - [10/Jan/2026:09:09:18 +0000] "GET / HTTP/1.1" 3
01 169 "-" "Wget"
162.142.125.46 - - [10/Jan/2026:09:15:32 +0000] "\x16\x03\x01\x00
\xEE\x01\x00\x00\xEA\x03\x03Pv3\xFD\x9A\xD3\x07\xE0\x15\xBE\xE8X\
x03\xBDk`\xB6T&O\xC0m\x13YD\xC8\x02\x09\x17\xC3(\x1D ,\xFES\x97\x
B4=\xC0\xAF\x14\x08\x9D\xCF\x91\x97\xAA\xBEnMW\x05\x19v$\xE8 \xCD
```

cat /etc/nginx/mime.types

```
[ec2-user@ip-10-0-10-128 ~]$ cat /etc/nginx/mime.types
types {
application/A2L                                  a2l;
application/AML                                  aml;
application/andrew-inset                         ez;
application/ATF                                  atf;
application/ATFX                                 atfx;
application/ATXML                                atxml;
application/atom+xml                             atom;
application/atomcat+xml                          atomcat;
application/atomdeleted+xml                      atomdeleted;
application/atomsvc+xml                          atomsvc;
application/atsc-dwd+xml                         dwd;
application/atsc-held+xml                        held;
application/atsc-rsat+xml                        rsat;
application/auth-policy+xml                      apxml;
application/bacnet-xdd+zip                       xdd;
application/calendar+xml                         xcs;
application/cbor                                 cbor;
application/cccex                                c3ex;
application/ccmp+xml                             ccmp;
application/ccxml+xml                            ccxml;
application/CDFX+XML                             cdfx;
application/cdmi-capability                      cdmia;
application/cdmi-container                       cdmic;
application/cdmi-domain                          cdmid;
application/cdmi-object                          cdmio;
application/cdmi-queue                           cdmiq;
application/CEA                                  cea;
application/cellml+xml                           cellml cml;
application/clue_info+xml                        clue;
application/cms                                  cmsc;
application/cpl+xml                              cpl;
```

cat /etc/ssl/certs/selfsigned. crt

```
[ec2-user@ip-10-0-10-128 ~]$ cat /etc/ssl/certs/selfsigned.crt
-----BEGIN CERTIFICATE-----
MIIDIDCCAgigAwIBAgIUMqRHSYrDB1uM4G6Ty/8ix355sTMwDQYJKoZIhvcNAQEL
BQAwFzEVMBMGA1UEAwwMNTEuMTEyLjU0LjgzMB4XDTI2MDEwMTAyNTg1OVoXDTI3
MDEwMTAyNTg1OVowFzEVMBMGA1UEAwwMNTEuMTEyLjU0LjgzMIIBIjANBgkqhkiG
9w0BAQEFAAOCAQ8AMIIBCgKCAQEAooxmQgnYqDo3N67ywMDAbFW9MhfVkNkPiYUl
F3LA3mkDdfxS8kPrToP5gh1BYWp2STZQ4Y6oP2Vx2Y/MBYd3HimwJECWbmeeBB9/
1wg7552jrAcdbBrCS9L6Wfn5111BVK86U1mY11UR9v2CD7lVC8ZcHKHHxl2ioDx7
giism864U6s4f7ikcmGjQqEHp4IooPTUKUexnxZBFRDhcuInTalDoUtOejZbYdiK
A/L+GPQnw4jZaTYCFnrnFjhqkHcLYmpnAyuRhFM65JcSMgD6L/UpcqyI0Qe139N8
l8UZCUUT7Ql0aQFdXA0AJZ1IPLc6Uv9h1vvqDUYq0r2JFdUnlQIDAQABo2QwYjAd
BgNVHQ4EFgQUjwPmFuWPrXFwpgLU9ebJ2HzUGMkwHwYDVR0jBBgwFoAUjwPmFuWP
rXFwpgLU9ebJ2HzUGMkwDwYDVR0TAQH/BAUwAwEB/zAPBgNVHREECDAGhwQzcDZT
MA0GCSqGSIb3DQEBCwUAA4IBAQAJdtOizTyP3E/1GRO8J2in4xOk07YF9x2kpQij
iOkovNUOJLiw0mxdbLxM7dDA5x0Ks0lCaMAF2N+U5XLXX1wZKRSmfUjdxg7igTEq
sJ+88/s/XH0PzZF2hTM1dSVlzwAGONSXW1QkofP8eisoSOUgY0ojFo0vXLW5iltv
V60LE7y8To73NLefYvfIiSkpVNOIG/PvhaJMXtJTrcR2PBml7OJu+RW+3xd+GurH
T+9TaEyM+WPRAkWcnHiLsHZydldBczgqiDQ0wcDCkJEucZBF6OKt3Aj/bXMBs3Sj
I2zocod6iZnMPKHQyTJWoU/H2xPHUzZObxL3sQ2O4CQYmtnU
-----END CERTIFICATE-----
[ec2-user@ip-10-0-10-128 ~]$
```

cat /etc/ssl/private/selfsigned.key

```
[ec2-user@ip-10-0-10-128 ~]$ sudo cat /etc/ssl/private/selfsigned.key
-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBKgwggSkAgEAAoIBAQCijGZCCdioOjc3
rvLAwMBsVb0yF9WQ2Q+JhSUXcsDeaQN1/FLyQ+tOg/mCHUFhanZJNlDhjqg/ZXHZ
j8wFh3ceKbAkQJZuZ54EH3/XCDvnnaOsBx1sGsJL0vpZ+fnXXUFUrzpTWZjXVRH2
/YIPuVULxlwcocfGXaKgPHuCKKybzrhTqzh/uKRyYaNCoQengiig9NQpR7GfFkEV
EOFy4idNqUOhS056Nlth2IoD8v4Y9CfDiNlpNgIWeucWOGqQdwtiamcDK5GEUzrk
lxIyAPov9SlyrIjRB7Xf03yXxRkJRRPtCXRpAV1cDQAlnUg8tzpS/2HW++oNRirS
vYkV1SeVAgMBAAECggEAM2RhdlKnofSZ/ax+CsxGalonUb2wY7YFEA09H29EJG2e
TwDidr9bl7zpN6apQ7QFxvr50n6omjaoKsmojzz3v90dWDDlfu2ay6Hr6OAtFHtG
Qr8TIdlKAfAoACelQt60p6IpNi4XQUmfvvAC3ZbSmUDzYYgS4hg7sR6+S/YxMKdH
JIg1QAEl3mPk8YfdXz3sDH5WoR9Oklzn32O6JcWvLpQS6LKSVmR+usdbWAuX43Od
hnOejm240XnY4eKQ3vrlJajvfOqmgiHfANPm7foj3y9TFHJogDhTYVpz4NHRt1nD
qt9ty3Hc3njeYvSfqqm8TuOIvgv+WDcCRn3I2FgAFwKBgQDfHLXYRxkxGfGstyTf
8WP3lKNOn5i5AIJFjg00r2R0Jg994DOBWkO+y7T5xOuprDb58aaosaDtjvPyi6Er
e9eX9JvrJmV2CHXlFbTxhg1VVZ6fbwRVkXgJpb1S8bdjDba/cDAASX7ov7swE2hv
O43IfqR0OuTaxAy3+40u3i0qHwKBgQC6gkahckKBkMQusVSYjCnG47pO7fmsOhby
qLrMhzN8Au8uobyHZni8L90muvL+keV2ZEzDgVjVpx3cIzYyBVYnXxQYhLM8dKNx
zt9Dp/dm1uz4Iu8DVZeY4hDos6ERRvVQbv+VwTVxhiCRXa0iNDBbiip24ll8vFMY
9voyWLUfywKBgB81Nd5lu4lZtqtRBtBoRHJqERm7SzJ9drth7kjzS5rPhBU4Whv6
00UJ22ugdtJJ63q0qXopNnhkKY1AqK+baAGyTmjq+wAymMMNcTzjYlQQYNquPa32
ZhL7YusKMnuhfHF0sNIpdZ36y6Ui4dXFP8TOqhQz9LUA/UJy5kGrBChDAoGBAIX6
5hA+S2ZV/4hnXRUEw1Ib74+ZxpyhUjDZYu9gRGzWksmV6CAATcUqQRz8eWjEl+kX
nklnltBMb9fe96SxTrWTyJTgZv2L8InmCV7Jv6EBzlNmjPqBNxjddTYlLBSEO9+f
DT2gAOtfZe/nMmN6yC5KL70dVxmTE8LAUPVv4hVHAoGBAJipA9HH8XMraCL9ROZe
hrw9Im8/C6ZPb9EhppA5izxo8IoiYo5oUfKJIGHLlqpi0xdDYEh1xCqMpjfEQ75B
K1suwOMNBqYHNBOeQkVfq0sE53Id88MjA0NIssM8buztG7g9yQ/5cPPYbn5Asw/+
HXdqn8EEf2MFHV3E+79GyaFg
-----END PRIVATE KEY-----
[ec2-user@ip-10-0-10-128 ~]$
```

Test reverse proxy in browser:

Open browser and navigate to https://<webserver-public-ip>

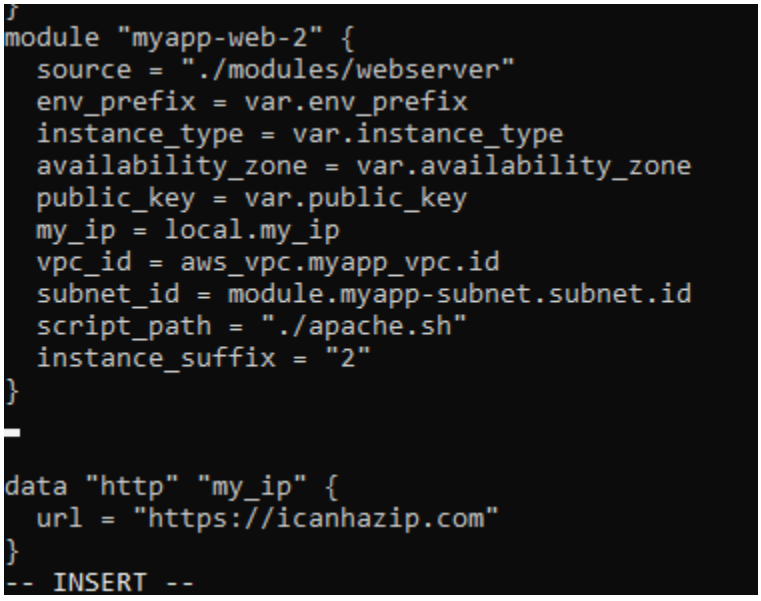
You should see the web-1 Apache page through the Nginx proxy

Task 8 — Configure Nginx as load balancer

In this task, you will add a second backend server and configure Nginx to load balance between them.

Add the second web server module to main.tf:

```
module "myapp-web-2" {
  source = "./modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var. availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
  vpc_id = aws_vpc.myapp_vpc. id
  subnet_id = module.myapp-subnet.subnet. id
  script_path = "./apache.sh"
  instance_suffix = "2"
}
```



Update outputs.tf:

```
output "aws_web-2_public_ip" {
  value = module. myapp-web-2.aws_instance.public_ip
```

}

```
}
output "aws_web-2_public_ip" {
  value = module.myapp-web-2.aws_instance.public_ip
}
```

Apply the configuration:

terraform apply -auto-approve

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform apply -au
to-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com
]
module.myapp-web-1.aws_key_pair.ssh_key: Refreshing state... [id=
dev-serverkey-1]
module.myapp-webserver.aws_key_pair.ssh_key: Refreshing state...
[id=dev-serverkey-0]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-03b2e88256558bdf9]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Refreshing st
ate... [id=igw-017f4f433aedd27a6]
module.myapp-web-1.aws_security_group.web_sg: Refreshing state...
 [id=sg-093bb3320fecccadc]
module.myapp-webserver.aws_security_group.web_sg: Refreshing stat
e... [id=sg-0a686729a9307cb13]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Refreshing state..
. [id=subnet-025e02d89c296eaaf]
module.myapp-subnet.aws_default_route_table.main_rt: Refreshing s
tate... [id=rtb-0d3fd7c1875b0bd17]
module.myapp-webserver.aws_instance.myapp_server: Refreshing stat
e... [id=i-0225475a01a1ae56f]
module.myapp-web-1.aws_instance.myapp_server: Refreshing state...
 [id=i-0b8cf3c3f1201f519]

Changes to Outputs:
  + aws_web_2_public_ip = (known after apply)
module.myapp-web-2.aws_key_pair.ssh_key: Creating...
module.myapp-web-2.aws_security_group.web_sg: Creating...
module.myapp-web-2.aws_key_pair.ssh_key: Creation complete after
0s [id=dev-serverkey-2]
module.myapp-web-2.aws_security_group.web_sg: Creation complete a
fter 2s [id=sg-0c13d0510b680def8]
module.myapp-web-2.aws_instance.myapp_server: Creating...
module.myapp-web-2.aws_instance.myapp_server: Still creating... [
00m10s elapsed]
module.myapp-web-2.aws_instance.myapp_server: Creation complete a
fter 13s [id=i-0063a6417b24ba47c]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

aws_web_1_public_ip = "51.112.45.114"
aws_web_2_public_ip = "40.172.221.93"
webserver_public_ip = "3.29.21.83"
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Get all outputs:

terraform output

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform output
aws_web_1_public_ip = "51.112.45.114"
aws_web_2_public_ip = "40.172.221.93"
webserver_public_ip = "3.29.21.83"
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

SSH into the webserver (Nginx proxy):

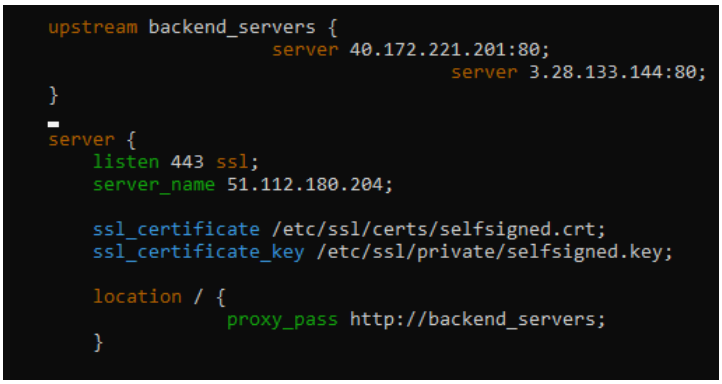
ssh ec2-user@<webserver-public-ip>

Edit Nginx configuration for load balancing:

sudo vim /etc/nginx/nginx.conf

Update the upstream block and location:

```
upstream backend_servers {
    server <web-1-public-ip>:80;
    server <web-2-public-ip>: 80;
}
# ...  in server block:
    location / {
#       root /usr/share/nginx/html;
#       index index.html;
#       proxy_pass http://<web-1-public-ip>:80;
        proxy_pass http://backend_server
    }
```



Restart Nginx:

sudo systemctl restart nginx

```
[ec2-user@ip-10-0-10-228 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-228 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enab
     Active: active (running) since Sat 2026-01-10 16:40:07 UTC;
    Process: 37170 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (c
    Process: 37171 ExecStartPre=/usr/sbin/nginx -t (code=exited,
    Process: 37172 ExecStart=/usr/sbin/nginx (code=exited, statu
   Main PID: 37173 (nginx)
      Tasks: 3 (limit: 1067)
     Memory: 3.3M
        CPU: 46ms
     CGroup: /system.slice/nginx.service
             ├─37173 "nginx: master process /usr/sbin/nginx"
             ├─37174 "nginx: worker process"
             └─37175 "nginx: worker process"

Jan 10 16:40:07 ip-10-0-10-228.me-central-1.compute.internal sys
Jan 10 16:40:07 ip-10-0-10-228.me-central-1.compute.internal ngi
Jan 10 16:40:07 ip-10-0-10-228.me-central-1.compute.internal ngi
Jan 10 16:40:07 ip-10-0-10-228.me-central-1.compute.internal ngi
Jan 10 16:40:07 ip-10-0-10-228.me-central-1.compute.internal ngi
Jan 10 16:40:07 ip-10-0-10-228.me-central-1.compute.internal sys
lines 1-21/21 (END)
```

Test load balancing in browser:

Open browser and navigate to https://<webserver-public-ip>

Reload the page multiple times

You should see the content alternating between web-1 and web-2 (check the hostname/IP in the page)

Task 9 — Configure high availability with backup servers

In this task, you will configure one server as primary and another as backup for high availability.

SSH into the webserver:

ssh ec2-user@<webserver-public-ip>

Edit Nginx configuration for high availability:

sudo vim /etc/nginx/nginx.conf

Update the upstream block to make web-2 a backup:

```
upstream backend_servers {
    server <web-1-public-ip>:80;
    server <web-2-public-ip>:80 backup;
}
```

```
http {
    include       /etc/nginx/mime.types;
    default_type  application/octet-stream;

    access_log  /var/log/nginx/access.log;
    error_log   /var/log/nginx/error.log notice;

    sendfile on;
    keepalive_timeout 65;

    upstream backend_servers {
        server 51.112.45.114:80;
        server 40.172.221.93:80;
    }

    server {
        listen 443 ssl;
        server_name _;

        ssl_certificate /etc/ssl/certs/selfsigned.crt;
        ssl_certificate_key /etc/ssl/private/selfsigned.key;
```

Restart Nginx:

sudo systemctl restart nginx

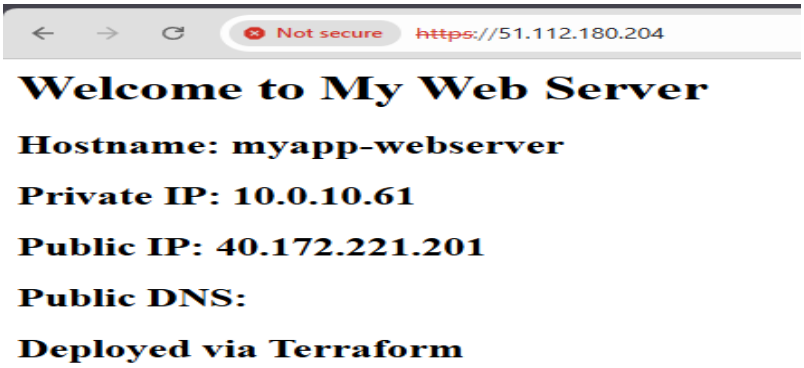
```
[ec2-user@ip-10-0-10-183 ~]$ sudo vim /etc/nginx/nginx.conf
[ec2-user@ip-10-0-10-183 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-183 ~]$
```

Test in browser:

Open browser and navigate to https://<webserver-public-ip>

Reload multiple times

You should ONLY see web-1 (primary server)

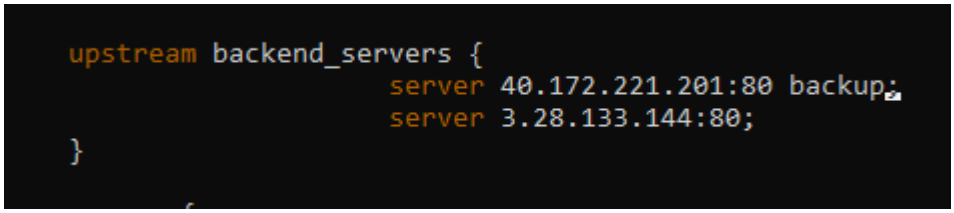


Switch backup configuration:

sudo vim /etc/nginx/nginx.conf

Update to make web-1 backup:

```
upstream backend_servers {
    server <web-1-public-ip>: 80 backup;
    server <web-2-public-ip>:80;
}
```



Restart Nginx:

sudo systemctl restart nginx

Test in browser:

Reload multiple times

You should ONLY see web-2 (now the primary server)

Task 10 — Enable Nginx caching

In this task, you will enable caching in Nginx to improve performance.

SSH into the webserver:

ssh ec2-user@<webserver-public-ip>

Edit Nginx configuration to enable caching:

sudo vim /etc/nginx/nginx.conf

Add proxy cache configuration in the http block and location block:

http {

proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=my_cache:10m inactive=60m max_size=1g;

log_format main '$remote_addr - $remote_user [$time_local] "$request" '

'$status $body_bytes_sent "$http_referer" '

'"$http_user_agent" "$http_x_forwarded_for"';

... other settings ...

upstream backend_servers {

server <web-1-public-ip>:80;

server <web-2-public-ip>: 80;

```
}
server {
listen 443 ssl;
server_name $PUBLIC_IP;
ssl_certificate /etc/ssl/certs/selfsigned.crt;
ssl_certificate_key /etc/ssl/private/selfsigned.key;
location / {
# root /usr/share/nginx/html;
# index index.html;
# proxy_pass http://<web-1-public-ip>: 80;
proxy_pass http://backend_servers;
proxy_cache my_cache;
proxy_cache_valid 200 60m;
proxy_cache_key "$scheme$request_uri";
add_header X-Cache-Status $upstream_cache_status;
}
}
# ... rest of config ...
}
```

```
http {
proxy_cache_path /var/cache/nginx
        levels=1:2
        keys_zone=my_cache:10m
        inactive=60m
        max_size=1g;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    sendfile on;
    keepalive_timeout 65;

    upstream backend_servers {
                        server 40.172.221.201:80 backup;
                        server 3.28.133.144:80;
    }

    server {
        listen 443 ssl;
        server_name 51.112.180.204;

        ssl_certificate /etc/ssl/certs/selfsigned.crt;
        ssl_certificate_key /etc/ssl/private/selfsigned.key;

        location / {
               proxy_pass http://backend_servers;
                   proxy_cache my_cache;
                       proxy_cache_valid 200 60m;
                           proxy_cache_key "$scheme$request_uri";
                               add_header X-Cache-Status $upstream_cache_status;
        }
```

Restart Nginx:

sudo systemctl restart nginx

```
[ec2-user@ip-10-0-10-183 ~]$ sudo vim /etc/nginx/nginx.conf
[ec2-user@ip-10-0-10-183 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-183 ~]$
```

Test caching in browser:

Open browser developer tools (F12)

Navigate to Network tab

Visit https://<webserver-public-ip>

Check response headers for X-Cache-Status

First request should show MISS

Reload the page

Second request should show HIT

```
[ec2-user@ip-10-0-10-183 ~]$ curl -Ik https://51.112.180.204
HTTP/1.1 200 OK
Server: nginx/1.28.0
Date: Thu, 01 Jan 2026 05:40:43 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 190
Connection: keep-alive
Last-Modified: Thu, 01 Jan 2026 04:41:03 GMT
ETag: "be-6474c333f7aca"
X-Cache-Status: HIT
Accept-Ranges: bytes
```

```
Accept-Ranges: bytes

[ec2-user@ip-10-0-10-183 ~]$ curl -Ik https://51.112.180.204
HTTP/1.1 200 OK
Server: nginx/1.28.0
Date: Thu, 01 Jan 2026 05:41:18 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 190
Connection: keep-alive
Last-Modified: Thu, 01 Jan 2026 04:41:03 GMT
ETag: "be-6474c333f7aca"
X-Cache-Status: HIT
Accept-Ranges: bytes
```

Verify cache directory:

ls -la /var/cache/nginx/

```
[ec2-user@ip-10-0-10-183 ~]$ sudo ls -la /var/cache/nginx/
total 0
drwx------. 3 nginx root   15 Jan  1 05:32 .
drwxr-xr-x. 9 root  root  101 Jan  1 05:31 ..
drwx------. 3 nginx nginx  16 Jan  1 05:32 4
[ec2-user@ip-10-0-10-183 ~]$
```

Cleanup

Exit SSH session:

exit

Destroy all resources:

terraform destroy

Type yes when prompted for confirmation.

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ terraform destroy
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.co
m]
module.myapp-webserver.aws_key_pair.ssh_key: Refreshing state...
 [id=dev-serverkey-0]
module.myapp-web-2.aws_key_pair.ssh_key: Refreshing state... [id
=dev-serverkey-2]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-03b2e88256558bdf9
]
module.myapp-web-1.aws_key_pair.ssh_key: Refreshing state... [id
=dev-serverkey-1]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Refreshing state.
.. [id=subnet-025e02d89c296eaaf]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Refreshing s
tate... [id=igw-017f4f433aedd27a6]
module.myapp-web-1.aws_security_group.web_sg: Refreshing state..
. [id=sg-093bb3320fecccadc]
module.myapp-webserver.aws_security_group.web_sg: Refreshing sta
te... [id=sg-0a686729a9307cb13]
module.myapp-web-2.aws_security_group.web_sg: Refreshing state..
. [id=sg-0c13d0510b680def8]
```

```
Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes
```

```
te after 41s
module.myapp-web-2.aws_key_pair.ssh_key: Destroying... [id=dev-s
erverkey-2]
module.myapp-web-2.aws_security_group.web_sg: Destroying... [id=
sg-0c13d0510b680def8]
module.myapp-webserver.aws_instance.myapp_server: Destruction co
mplete after 41s
module.myapp-webserver.aws_key_pair.ssh_key: Destroying... [id=d
ev-serverkey-0]
module.myapp-webserver.aws_security_group.web_sg: Destroying...
[id=sg-0a686729a9307cb13]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destroying... [id
=subnet-025e02d89c296eaaf]
module.myapp-web-2.aws_key_pair.ssh_key: Destruction complete af
ter 0s
module.myapp-webserver.aws_key_pair.ssh_key: Destruction complet
e after 0s
module.myapp-web-2.aws_security_group.web_sg: Destruction comple
te after 1s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destruction compl
ete after 1s
module.myapp-webserver.aws_security_group.web_sg: Destruction co
mplete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-03b2e88256558bdf9]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 13 destroyed.
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

Verify state files:

cat terraform.tfstate

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ cat terraform.tfst
ate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 59,
  "lineage": "219fbcb2-a55e-8935-db99-42e262f75dd6",
  "outputs": {},
  "resources": [],
  "check_results": null
}
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```

List all project files:

tree

or

ls –la

```
@manahilhabib031 ➜ /workspaces/Lab12 (main) $ ls -la
total 96
drwxrwxrwx+ 5 codespace root       4096 Jan 11 02:28 .
drwxr-xrwx+ 5 codespace root       4096 Jan 10 06:35 ..
drwxrwxrwx+ 8 codespace root       4096 Jan 10 06:37 .git
drwxrwxrwx+ 4 codespace codespace  4096 Jan 10 08:47 .terraform
-rw-r--r--  1 codespace codespace  2452 Jan 10 08:47 .terraform.
lock.hcl
-rw-rw-rw-  1 codespace root         11 Jan 10 06:34 README.md
-rw-rw-rw-  1 codespace codespace   852 Jan 10 15:40 apache.sh
-rwxrwxrwx  1 codespace codespace  1573 Jan 10 08:53 entry-scrip
t.sh
-rw-rw-rw-  1 codespace codespace   124 Jan 10 08:34 locals.tf
-rw-rw-rw-  1 codespace codespace  1851 Jan 10 16:16 main.tf
drwxrwxrwx+ 4 codespace codespace  4096 Jan 10 08:41 modules
-rw-rw-rw-  1 codespace codespace   261 Jan 10 16:17 outputs.tf
-rw-rw-rw-  1 codespace codespace   182 Jan 11 02:28 terraform.t
fstate
-rw-rw-rw-  1 codespace codespace 35323 Jan 11 02:28 terraform.t
fstate.backup
-rw-rw-rw-  1 codespace codespace   254 Jan 10 08:34 terraform.t
fvars
-rw-rw-rw-  1 codespace codespace   198 Jan 10 08:34 variables.t
f
@manahilhabib031 ➜ /workspaces/Lab12 (main) $
```